

ACTUARIAL PORTFOLIO PROJECT · RESERVING

Stochastic Reserving

End-to-end reserve estimation from deterministic chain ladder to stochastic bootstrapping, including Mack theory, ODP Bootstrap, VaR, CoV, and IFRS 17 implications at Senior Actuarial Analyst level.

DIFFICULTY

Senior Analyst

DOMAIN

Non-Life Reserving

METHODS

CL · BF · Mack · ODP Bootstrap

TOOLS

Python · R · Power BI

- Chain Ladder
- Bornhuetter-Ferguson
- Mack Model
- Bootstrap ODP
- VaR · TVaR · CoV
- IFRS 17
- GLM Reserving
- ChainLadder (R)
- Reserve Risk

00

NAVIGATION

Table of Contents

01	Business Context	02	Project Objectives & KPIs
03	Data Requirements	04	Synthetic Triangle Generation
05	Triangle Diagnostics & EDA	06	Deterministic Methods (CL & BF)
07	Mack Chain Ladder — Full Theory	08	Bootstrap ODP — Full Code
09	Reserve Distribution & Risk Measures	10	Model Validation & Diagnostics
11	Business Interpretation	12	Power BI Dashboard Design
13	80 Interview Questions & Answers	14	Portfolio & GitHub Structure

01

CONTEXT

Business Context

What is Claims Reserving?

Claims reserving is the process of **estimating the total amount an insurer will ultimately pay on claims that have already occurred** but have not yet been fully settled. These outstanding liabilities sit on the insurer's balance sheet and are the largest source of uncertainty in non-life (general) insurance financial statements.

Reserves are not simply an accounting estimate — they drive solvency ratios, reinsurance pricing, executive bonuses, IRDAI/FCA regulatory returns, and, under IFRS 17, the Liability for Incurred Claims (LIC) which directly impacts the income statement.

The Two Reserve Components

COMPONENT	DESCRIPTION	STOCHASTIC IMPORTANCE
IBNER — Incurred But Not Enough Reserved	Claims already reported but case reserves are insufficient	Main driver of reserve development on older accident years
IBNR — Incurred But Not Reported	Claims that have occurred but not yet notified to the insurer	Dominant uncertainty on recent accident years; drives stochastic spread

Why Stochastic Reserving?

Deterministic methods (chain ladder, BF) give a **single-point best estimate**. They answer "what is our best guess?" but not "how uncertain are we?" Stochastic reserving answers:

- What is the **probability distribution** of ultimate claims?
- What reserve do we need to be 75% / 90% confident we don't run short?
- What is the **Coefficient of Variation** — the relative uncertainty around the best estimate?
- What is the **Value at Risk** at the 99.5th percentile — the Solvency II capital requirement?
- How does reserve uncertainty contribute to the **Risk Margin** under IFRS 17?

THE BUSINESS PROBLEM

ShieldRe General Insurance (synthetic motor third-party liability insurer) has carried ₹420Cr of case reserves on its TP motor book. The board and Chief Actuary need to know: (1) Is the carried reserve adequate? (2) What additional buffer should be held at 75th and 99.5th percentile? (3) How much of the technical provisions under IFRS 17 should be attributable to reserve risk in the Risk Adjustment?

Regulatory Context — India & International

FRAMEWORK	REQUIREMENT	ROLE OF STOCHASTIC RESERVING
IRDAI (India)	IBNR must be certified by Appointed Actuary; discounting not permitted	Stochastic provides the uncertainty range around the best estimate
IFRS 17	Liability for Incurred Claims (LIC) = Best Estimate Liabilities + Risk Adjustment	Risk Adjustment quantified at a specified confidence level — stochastic distribution required
Solvency II / IRDAI RBC	SCR Reserve Risk module: 1-in-200 year loss (99.5th percentile)	Bootstrap distribution directly inputs SCR calculation
IFoA/CAS	Best practice: TAS 200 (UK), ASOP 43 (US) — uncertainty disclosure required	Full stochastic analysis satisfies both professional standards

02 OBJECTIVES

Project Objectives & KPIs

CORE ACTUARIAL QUESTION

What is the full probability distribution of outstanding claims liabilities for ShieldRe's motor TP book, and what reserve should management hold to be 75% and 99.5% confident of adequacy?

Deliverables

1. A 10×10 paid and incurred claims development triangle (accident years 2014–2023)
2. Deterministic best estimates: Chain Ladder, Bornhuetter–Ferguson, Volume–Weighted Average
3. Mack standard error by accident year with 95% confidence intervals
4. Bootstrap ODP simulation: 10,000 reserve scenarios
5. Reserve distribution: histogram, VaR(75%), VaR(90%), VaR(99.5%), TVaR(99.5%), CoV
6. Diagnostic checks: residual plots, development factor stability, back-testing
7. IFRS 17 Risk Adjustment quantification
8. Power BI monitoring dashboard specification



03 SPECIFICATION

Data Requirements

Claims Development Triangle

The primary input is a **claims development triangle**: a matrix where rows represent accident years (or underwriting years) and columns represent development periods (age at which data is measured). Cell (i,j) = cumulative paid claims for accident year i at development period j.

CRITICAL TERMINOLOGY – KNOW THESE COLD

- **Accident year (AY):** year in which the insured event occurred

- **Underwriting year (UWY):** year in which the policy was written — more complex development pattern
- **Development period:** time since start of accident year (1yr = 12 months development)
- **Cumulative paid claims:** total cash paid to claimants up to the development age
- **Cumulative incurred claims:** paid + case reserves (IBNER) — develops more smoothly
- **Incremental claims:** claims paid in each specific period (triangle can be in either form)
- **Latest diagonal:** the most recent observations — the set of data points at the edge of known information

Data Inputs Required

DATASET	FORMAT	REQUIRED COLUMNS	NOTES
Paid Claims Triangle	10×10 matrix (₹ lakhs)	Accident Year, Dev 1–10	Primary input for Chain Ladder
Incurred Claims Triangle	10×10 matrix (₹ lakhs)	Same structure	Used for Bornhuetter–Ferguson a priori
Earned Premium by AY	Vector (10 values)	AY, Earned Premium ₹	Required for BF loss ratio a priori
A Priori Loss Ratios	Vector (10 values)	AY, Expected LR%	Set by pricing actuary; BF input
Case Reserves	Vector (latest diagonal)	AY, Case Reserve ₹	For IBNR vs IBNER split
Claim Counts Triangle	10×10 optional	Same structure	Used for frequency–severity reserving check

Data Quality Checklist

1. **Consistency of definition:** Confirm paid means settled-and-closed, not partially paid. Confirm incurred includes all case reserve movements.
2. **Large losses:** Identify claims above ₹1Cr — may need separate treatment. Include/exclude consistently across years.

3. **Reinsurance netting:** Ensure triangle is on gross or net basis consistently — mixing distorts development patterns.
4. **Claims handling changes:** Any changes in reserving philosophy (e.g., new claims management system in 2019) may create artificial kinks in development patterns.
5. **Portfolio changes:** New business volumes, product mix changes, or geographic expansion may make older years a poor guide to future development.
6. **Currency:** Ensure all values are in the same unit (₹ lakhs) and not inflation-adjusted.

04 PYTHON CODE Synthetic Triangle Generation

We generate a realistic 10×10 claims development triangle that mimics motor TP liability characteristics: slow development, long tail, genuine process noise, and one anomalous accident year (2017 — adverse development from court ruling change).

PYTHON

```
# =====
# Stochastic Reserving – Synthetic Triangle Generation
# Motor TP Liability | 10 Accident Years × 10 Dev Periods
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(2024)

# — True development factors ( $\lambda_1, \lambda_2, \dots, \lambda_9$ ) —————
# Motor TP: slow to develop (tail heavy)
TRUE_CDF = np.array([
    3.20, # 12→24 months: still only 31% developed at 12m
    1.75, # 24→36 months
    1.35, # 36→48
    1.18, # 48→60
```

```

1.09, # 60→72
1.05, # 72→84
1.03, # 84→96
1.015, # 96→108
1.008, # 108→120 (tail)
])

# Ultimate loss ratio by accident year (true underlying risk)
TRUE_ULT_LR = np.array([
    0.72, 0.74, 0.76, 0.88, # 2014-2017: 2017 = court ruling shock
    0.75, 0.77, 0.79, 0.80, # 2018-2021: moderate trend
    0.82, 0.84                # 2022-2023: recent elevated risk
])

# Earned premium by accident year (₹ Lakhs)
PREMIUM = np.array([
    5800, 6200, 6800, 7200,
    7800, 8500, 9200, 9800,
    10500, 11200
])

TRUE_ULTIMATE = TRUE_ULT_LR * PREMIUM # True ultimate by AY
N_AY = 10 # 10 accident years (2014-2023)
N_DEV = 10 # 10 development periods

# — Compute cumulative payment percentages —————
# % paid by development period = 1 / product of remaining CDFs
pct_paid = np.ones(N_DEV)
cdf_to_ultimate = np.ones(N_DEV)
running_cdf = 1.0
for j in range(N_DEV - 1, -1, -1):
    if j == N_DEV - 1:
        cdf_to_ultimate[j] = 1.0
    else:
        cdf_to_ultimate[j] = cdf_to_ultimate[j+1] * TRUE_CDF[j]

pct_paid = 1 / cdf_to_ultimate

```

```

# — Generate cumulative paid triangle with noise —————
# Add overdispersed Poisson noise:  $Var = \phi \times \mu$  (ODP assumption)
# This is the data-generating process consistent with the bootstrap model
DISP_PARAM = 250 #  $\phi$  (overdispersion – typical for TP motor)

paid_triangle = np.zeros((N_AY, N_DEV))
incurred_triangle = np.zeros((N_AY, N_DEV))

for i in range(N_AY):
    prev = 0.0
    for j in range(N_DEV):
        mean_incr = TRUE_ULTIMATE[i] * (pct_paid[j] - (0 if j == 0 else pct_paid[j-1]))
        mean_incr = max(mean_incr, 0.1)
        # ODP: variance =  $\phi \times mean \rightarrow$  simulate via Gamma( $mean/\phi$ ,  $\phi$ )
        incr = np.random.gamma(mean_incr / DISP_PARAM, DISP_PARAM)
        paid_triangle[i, j] = prev + incr
        prev = paid_triangle[i, j]

    # Incurred = paid + case reserve (mean = remaining IBNR)
    # Case reserve developments smoothly toward ultimate
    for j in range(N_DEV):
        remaining = TRUE_ULTIMATE[i] - paid_triangle[i, j]
        case_noise = np.random.normal(0, remaining * 0.05) if remaining > 0 else 0
        incurred_triangle[i, j] = paid_triangle[i, j] + max(0, remaining + case_noise)

# — 2017 anomaly: court ruling shock —————
# Accident year 2017 (index 3): unexpected +15% reserve strengthening
# at development 48-72 months due to retroactive court award change
paid_triangle[3, 3:] *= 1.15
incurred_triangle[3, 2:] *= 1.12

# — Apply calendar year diagonal mask —————
# Only cells where  $i + j < N\_AY$  are observable (upper triangle)
# Latest diagonal = evaluation date (Dec 2023)
paid_observed = paid_triangle.copy()
incurred_observed = incurred_triangle.copy()

for i in range(N_AY):

```



```

for j in range(N_DEV):
    if i + j >= N_AY:
        paid_observed[i, j] = np.nan
        incurred_observed[i, j] = np.nan

# — Format as DataFrames —————
AY_LABELS = list(range(2014, 2024))
DEV_LABELS = [f'{(j+1)*12}m' for j in range(N_DEV)]

paid_df = pd.DataFrame(paid_observed.round(1),
                        index=AY_LABELS, columns=DEV_LABELS)
incurred_df = pd.DataFrame(incurred_observed.round(1),
                            index=AY_LABELS, columns=DEV_LABELS)
premium_df = pd.Series(PREMIUM, index=AY_LABELS, name='earned_premium')

# — A Priori Loss Ratios (from pricing team) —————
# Used for Bornhuetter-Ferguson – set conservatively by pricing
APRIORI_LR = pd.Series([
    0.74, 0.75, 0.76, 0.75, 0.76,
    0.77, 0.78, 0.78, 0.79, 0.80
], index=AY_LABELS, name='apriori_lr')

paid_df.to_csv('paid_triangle.csv')
incurred_df.to_csv('incurred_triangle.csv')
premium_df.to_csv('premium.csv')

print("=== PAID CLAIMS TRIANGLE (₹ Lakhs) ===")
print(paid_df.to_string(na_rep='---'))
print(f"\nTrue ultimates: {TRUE_ULTIMATE.round(1)}")
print(f"Portfolio total ultimate: ₹{TRUE_ULTIMATE.sum():.1f} Lakhs")

```

The Paid Claims Triangle (₹ Lakhs)

A representative output of the synthetic triangle. The lower-right is unobserved (future development). Latest diagonal = December 2023 valuation date.

AY	12M	24M	36M	48M	60M	72M	84M	96M	108M	120M
2014	1,318	2,204	2,868	3,287	3,541	3,708	3,822	3,895	3,940	3,972
2015	1,448	2,389	3,107	3,552	3,831	3,985	4,088	4,152	4,192	—
2016	1,594	2,610	3,396	3,892	4,189	4,368	4,483	4,556	—	—
2017	1,821	3,003	4,038	4,756	5,113	5,336	5,485	—	—	—
2018	1,782	2,958	3,843	4,405	4,757	4,966	—	—	—	—
2019	1,975	3,228	4,193	4,795	5,173	—	—	—	—	—
2020	2,048	3,420	4,438	5,091	—	—	—	—	—	—
2021	2,312	3,752	4,888	—	—	—	—	—	—	—
2022	2,488	4,032	—	—	—	—	—	—	—	—
2023	2,682	—	—	—	—	—	—	—	—	—

Dark cells = observed data. Empty cells = future development to be projected. Triangle is upper-left.

05

EDA

Triangle Diagnostics & EDA

PYTHON

```
# =====  
  
# Triangle Diagnostics – Full Actuarial EDA  
  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import matplotlib.gridspec as gridspec  
import seaborn as sns  
  
paid = pd.read_csv('paid_triangle.csv', index_col=0)  
incurred = pd.read_csv('incurred_triangle.csv', index_col=0)  
premium = pd.read_csv('premium.csv', index_col=0).squeeze()  
  
TEAL='#1a8a7a'; AMBER='#d4882a'; INK='#0d0f14'
```

```

# — 1. INCREMENTAL CLAIMS —————
# Incremental = Cumulative[j] - Cumulative[j-1]
# Should be positive – negatives indicate reserve releases / corrections
incremental = paid.diff(axis=1)
incremental.iloc[:, 0] = paid.iloc[:, 0] # first dev = cumulative
neg_count = (incremental < 0).sum().sum()
print(f"Negative incremental cells: {neg_count} (ideally 0 for motor TP)")

# — 2. VOLUME-WEIGHTED LINK RATIOS —————
#  $f_j = \sum_i C(i, j) / \sum_i C(i, j-1)$  for all observed pairs
def compute_link_ratios(cum_tri):
    """Compute age-to-age development factors (link ratios)"""
    tri = cum_tri.values.copy()
    n_ay, n_dev = tri.shape
    link_ratios = pd.DataFrame(index=cum_tri.index, columns=cum_tri.columns[1:])

    for j in range(1, n_dev):
        for i in range(n_ay):
            if not (np.isnan(tri[i, j]) or np.isnan(tri[i, j-1]) or tri[i, j-1] == 0):
                link_ratios.iloc[i, j-1] = tri[i, j] / tri[i, j-1]

    return link_ratios.astype(float)

ldf_indiv = compute_link_ratios(paid)

# Volume-weighted average LDF (the standard chain ladder factor)
ldf_vwa = {}
tri_vals = paid.values
for j in range(1, paid.shape[1]):
    num = 0; den = 0
    for i in range(paid.shape[0]):
        if not (np.isnan(tri_vals[i, j]) or np.isnan(tri_vals[i, j-1])):
            num += tri_vals[i, j]; den += tri_vals[i, j-1]
    ldf_vwa[paid.columns[j]] = num / den if den > 0 else np.nan

print("\nVolume-Weighted Average Link Ratios:")
for k, v in ldf_vwa.items():
    print(f" {k}: {v:.4f}")

```

```

# — 3. LINK RATIO STABILITY PLOTS —————
# Plot individual LDFs by development period – check for outliers and trends
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
fig.suptitle('Development Factor Stability by Development Period',
             fontsize=13, fontweight='bold', color=INK)
axes = axes.flatten()

for idx, col in enumerate(ldf_indiv.columns[:8]):
    ax = axes[idx]
    col_data = ldf_indiv[col].dropna()
    ax.bar(col_data.index, col_data.values, color=TEAL, alpha=0.7)
    ax.axhline(ldf_vwa.get(col, np.nan), color=AMBER, linewidth=2,
              linestyle='--', label=f'VWA: {ldf_vwa.get(col,0):.3f}')
    ax.set_title(f'{col} LDF', fontsize=9, color=INK, fontweight='bold')
    ax.tick_params(axis='x', rotation=45, labels=7)
    ax.legend(fontsize=7)

plt.tight_layout()
plt.savefig('ldf_stability.png', dpi=150, bbox_inches='tight')

# — 4. LOSS RATIO DEVELOPMENT —————
# Plot incurred LR by AY at each development age
# Should converge toward stable ultimate – divergence signals problem
fig, ax = plt.subplots(figsize=(11, 5))
for i, ay in enumerate(paid.index):
    lr_dev = (paid.loc[ay] / premium.loc[ay]).dropna()
    ax.plot(range(1, len(lr_dev)+1), lr_dev.values,
           'o-', markersize=4, linewidth=1.5,
           label=str(ay), alpha=0.8)

ax.set_xlabel('Development Period (years)')
ax.set_ylabel('Paid Loss Ratio')
ax.set_title('Paid Loss Ratio Development by Accident Year',
            color=INK, fontweight='bold')
ax.legend(loc='upper left', fontsize=8, ncol=2)
plt.tight_layout()
plt.savefig('lr_development.png', dpi=150, bbox_inches='tight')

```

```

# — 5. CALENDAR YEAR DIAGONAL ANALYSIS —————
# Check for diagonal (calendar year) effects: claims inflation,
# change in settlement speed, or external shock on a specific year
def calendar_year_effects(tri):
    """Clark-Venter diagonal test: compute average LDF for each calendar year"""
    n_ay, n_dev = tri.shape
    cal_effects = {}
    for d in range(1, n_dev):
        ldfs = []
        for i in range(n_ay):
            if i + d < n_ay and not (np.isnan(tri[i,d]) or np.isnan(tri[i,d-1])):
                ldfs.append(tri[i,d] / tri[i,d-1])
        cal_effects[2014+d] = np.mean(ldfs) if ldfs else np.nan
    return cal_effects

cy_effects = calendar_year_effects(paid.values)
print("\nCalendar Year Diagonal Effects (avg LDF):")
for cy, v in cy_effects.items():
    if not np.isnan(v):
        print(f"  CY {cy}: {v:.4f}")

# — 6. PAID vs INCURRED COMPARISON —————
# Incurred/Paid ratio > 1 means case reserves are being set
# Decreasing ratio = rapid settlement pace; increasing = slow settlement
latest_diag_paid = [paid.iloc[i, -1-i] for i in range(paid.shape[0]-1, -1, -1)]
latest_diag_paid.reverse()
# Extract latest diagonal from each row
latest_paid = paid.apply(lambda row: row.dropna().iloc[-1], axis=1)
latest_incurred = incurred.apply(lambda row: row.dropna().iloc[-1], axis=1)
ip_ratio = latest_incurred / latest_paid
print("\nIncurred/Paid Ratio at Latest Diagonal:")
print(ip_ratio.round(3))
print("Ratios close to 1 = well-developed AYs (old); high ratios = immature AYs (recent)

```

Diagnostic Summary — What to Look For

DIAGNOSTIC	WHAT TO CHECK	RED FLAG
LDF stability plot	Individual factors should cluster around VWA	One AY consistently above/below — rogued year; should be excluded or down-weighted
Negative incrementals	Should be near zero for TP motor	Multiple negatives → reserve releases or data errors; BF or Cape Cod preferred
LR development chart	Lines should converge as development progresses	Lines diverge → heterogeneous AYs; consider splitting by sub-class
Calendar year effects	No systematic pattern across diagonals	High CY2019/2020 LDF → COVID claims surge or inflation shock
Incurred/Paid ratio	Old AYs → close to 1; recent → higher	Ratio rising for old AYs → IBNER emerging — case reserves deficient

06 METHODS Deterministic Methods — Chain Ladder & BF

PYTHON

```
# =====
# Deterministic Reserving – Chain Ladder & Bornhuetter-Ferguson
# =====

import pandas as pd
import numpy as np

paid = pd.read_csv('paid_triangle.csv', index_col=0)
premium = pd.read_csv('premium.csv', index_col=0).squeeze()
apriori_lr = pd.Series([0.74, 0.75, 0.76, 0.75, 0.76,
                        0.77, 0.78, 0.78, 0.79, 0.80],
                        index=paid.index, name='apriori_lr')

# — CHAIN LADDER —————
def chain_ladder(paid_tri):
    """
    Standard Volume-Weighted Chain Ladder.
```

```

Development factors  $\hat{f}_j = \sum_i C(i, j+1) / \sum_i C(i, j)$ 
Projected ultimate =  $C(i, \text{latest}) \times n(\hat{f}_j \text{ for remaining periods})$ 
"""

tri = paid_tri.values.copy().astype(float)
n_ay, n_dev = tri.shape

# Step 1: Compute VW link ratios
ldf = np.ones(n_dev - 1)
for j in range(n_dev - 1):
    num = den = 0.0
    for i in range(n_ay):
        if not (np.isnan(tri[i, j]) or np.isnan(tri[i, j+1])):
            num += tri[i, j+1]
            den += tri[i, j]
    ldf[j] = num / den if den > 0 else 1.0

# Step 2: Tail factor (add 1.0 at ultimate)
# For 10yr development with motor TP, tail beyond 120m ≈ 1.005
TAIL = 1.005
ldf_with_tail = np.append(ldf, TAIL)

# Step 3: Cumulative development factors (CDF-to-ultimate)
cdf = np.ones(n_dev)
cdf[-1] = TAIL
for j in range(n_dev - 2, -1, -1):
    cdf[j] = cdf[j+1] * ldf[j]

# Step 4: Project each AY to ultimate
projected = tri.copy()
ultimates = np.zeros(n_ay)
latest_dev = np.zeros(n_ay, dtype=int)

for i in range(n_ay):
    # Find latest observed development period
    observed = ~np.isnan(tri[i, :])
    last_j = np.where(observed)[0].max()
    latest_dev[i] = last_j

```

```

    # Project forward
    for j in range(last_j + 1, n_dev):
        projected[i, j] = projected[i, j-1] * ldf[j-1]

    ultimates[i] = projected[i, n_dev-1] * TAIL

# Step 5: IBNR = Ultimate - Latest Paid
latest_paid = np.array([tri[i, latest_dev[i]] for i in range(n_ay)])
ibnr = ultimates - latest_paid

return pd.DataFrame({
    'latest_paid':    latest_paid,
    'cl_ultimate':    ultimates.round(1),
    'cl_ibnr':        ibnr.round(1),
    'dev_age':        (latest_dev + 1) * 12,
    'pct_developed': (1 / cdf[latest_dev] * 100).round(1),
}, index=paid_tri.index), ldf, cdf

cl_results, ldf, cdf = chain_ladder(paid)
print("=== CHAIN LADDER RESULTS ===")
print(cl_results.to_string())
print(f"\nTotal CL IBNR: ₹{cl_results['cl_ibnr'].sum():.1f} Lakhs")

# — BORNHUETTER-FERGUSON —————
def bornhuetter_ferguson(paid_tri, premium, apriori_lr, ldf, cdf):
    """
    BF Method: Reserve = Expected Ultimate × (1 - 1/CDF)
    Where Expected Ultimate = Premium × A Priori LR

    BF blends the a priori expected loss with the observed paid loss:
    BF Ultimate = Paid + Expected × (1 - % Paid)
    = C(i,j) + Q(i) × (1 - 1/CDF(j))
    Where Q(i) = a priori ultimate = Premium(i) × ELR
    """
    tri = paid_tri.values.copy().astype(float)
    n_ay = tri.shape[0]
    n_dev = tri.shape[1]

```



```

apriori_ult = premium.values * apriori_lr.values

results = []
for i in range(n_ay):
    observed = ~np.isnan(tri[i, :])
    last_j = np.where(observed)[0].max()
    latest_paid = tri[i, last_j]

    # % unreported = 1 - 1/CDF = residual unreported factor
    pct_unreported = 1 - 1 / cdf[last_j]

    # BF IBNR = a priori ultimate * % unreported
    bf_ibnr = apriori_ult[i] * pct_unreported
    bf_ultimate = latest_paid + bf_ibnr

    results.append({
        'latest_paid':    latest_paid,
        'apriori_ult':    apriori_ult[i].round(1),
        'pct_unreported': (pct_unreported * 100).round(1),
        'bf_ibnr':        bf_ibnr.round(1),
        'bf_ultimate':    bf_ultimate.round(1),
    })

return pd.DataFrame(results, index=paid_tri.index)

bf_results = bornhuetter_ferguson(paid, premium, apriori_lr, ldf, cdf)
print("\n=== BORNHUETTER-FERGUSON RESULTS ===")
print(bf_results.to_string())
print(f"\nTotal BF IBNR: ₹{bf_results['bf_ibnr'].sum():,.1f} Lakhs")

# — COMPARISON TABLE —————
comparison = pd.DataFrame({
    'Latest Paid':    cl_results['latest_paid'].round(1),
    'CL Ultimate':    cl_results['cl_ultimate'],
    'CL IBNR':        cl_results['cl_ibnr'],
    'BF Ultimate':    bf_results['bf_ultimate'],
    'BF IBNR':        bf_results['bf_ibnr'],
    'CL vs BF Δ':     (cl_results['cl_ultimate'] - bf_results['bf_ultimate']).round(1),

```

```
} , index=paid.index)
print("\n=== CL vs BF COMPARISON ===")
print(comparison.to_string())
print(f"\nCL-BF difference: ₹{comparison['CL vs BF Δ'].sum():,.1f} Lakhs")
print("Positive = CL picks up higher IBNR; typical for immature recent AYs")
```

Chain Ladder Formula — The Core Mathematics

$$\hat{f}_j = \Sigma_i C(i, j+1) / \Sigma_i C(i, j)$$
 [Volume-Weighted Link Ratio]

$$C(i, j+1) = C(i, j) \times \hat{f}_j$$
 [Projection]

$$\hat{C}(i, \infty) = C(i, J) \times \hat{f}_j \times \hat{f}_{j+1} \times \dots \times \hat{f}_{n-1} \times (\text{tail factor})$$

$$IBNR(i) = \hat{C}(i, \infty) - C(i, J)$$
 [Outstanding Reserve]

WHERE C(I, J) = CUMULATIVE PAID CLAIMS, AY I, DEVELOPMENT PERIOD J. J = LAST OBSERVED PERIOD FOR AY I.

Bornhuetter-Ferguson Formula

$$\hat{q}_j = 1 - 1 / CDF_j$$
 [Expected % Unreported]

$$Q(i) = \text{Premium}(i) \times ELR(i)$$
 [A Priori Expected Ultimate]

$$IBNR_BF(i) = Q(i) \times \hat{q}_j$$
 [BF Reserve]

$$Ult_BF(i) = C(i, J) + Q(i) \times \hat{q}_j$$
 [BF Ultimate]

BF IS A CREDIBILITY BLEND: FULLY UNDEVELOPED AYS → 100% PRIOR. FULLY DEVELOPED → 100% OBSERVED. PARTICULARLY STABLE FOR IMMATURE AYS WHERE CL IS VOLATILE.

WHEN TO USE WHICH METHOD

METHOD	BEST FOR	WEAKNESS
Chain Ladder	Mature AYS (>60% developed); stable development patterns	Amplifies any accident year volatility forward; poor for immature years
Bornhuetter-Ferguson	Immature AYS; sparse recent-year data; when a priori is reliable	Sensitive to the choice of a priori LR; can suppress emerging trends

METHOD	BEST FOR	WEAKNESS
Cape Cod / ELR	When all AYs are thin; emerging lines of business	Ignores any information in the observed paid triangle
Average of CL + BF	Transition years — some credibility but not fully developed	Ad hoc weighting; not statistically optimal

07

STOCHASTIC THEORY

Mack Chain Ladder — Full Mathematical Theory & SE Derivation

The Mack (1993) model is the **distribution-free stochastic framework for the chain ladder**. It derives the mean squared error (MSE) of the chain ladder reserve without assuming a specific parametric distribution — hence "distribution-free." It is the most widely used method for stochastic reserving in UK and Indian practice.

The Three Mack Assumptions

CORE ASSUMPTIONS — MUST MEMORISE

Mack's three assumptions are often tested as a standalone interview question.

M1: $E[C(i, j+1) \mid C(i,1), \dots, C(i,j)] = C(i,j) \times f(j)$

EXPECTED FUTURE DEVELOPMENT DEPENDS ONLY ON THE LATEST CUMULATIVE (MARKOV PROPERTY). THE EXPECTED DEVELOPMENT FACTOR $f(j)$ IS THE SAME FOR ALL ACCIDENT YEARS.

M2: $Var[C(i, j+1) \mid C(i,1), \dots, C(i,j)] = C(i,j) \times \sigma^2(j)$

VARIANCE IS PROPORTIONAL TO CURRENT CUMULATIVE CLAIMS (OVERDISPERSED POISSON STRUCTURE). $\sigma^2(j)$ IS A PERIOD-SPECIFIC VARIANCE PARAMETER.

M3: $\{C(i,1), \dots, C(i,n)\}, i = 1, \dots, n$ are independent across accident years

DIFFERENT ACCIDENT YEAR DEVELOPMENTS ARE INDEPENDENT. CALENDAR YEAR CORRELATION (E.G., INFLATION AFFECTING A WHOLE DIAGONAL) WOULD VIOLATE THIS.

Variance Parameter Estimation

$$\sigma^2(j) = 1/(n-j-1) \times \sum_i C(i,j) \times [C(i,j+1)/C(i,j) - \hat{f}(j)]^2$$

WEIGHTED SQUARED DEVIATION OF INDIVIDUAL LDFS FROM THE VW AVERAGE. $N-J-1$ DEGREES OF FREEDOM CORRECTION. Σ^2 FOR THE LAST PERIOD ESTIMATED BY EXTRAPOLATION.

MSE of the Chain Ladder Reserve

The mean squared error of the IBNR for a single accident year i is:

$$MSE(\hat{C}(i,n)) = \hat{C}(i,n)^2 \times \sum_{j=a(i)}^{n-1} [\sigma^2(j)/\hat{f}^2(j)] \times [1/C(i,j) + 1/\sum_k C(k,j)]$$

FIRST TERM: PROCESS VARIANCE (RANDOMNESS IN FUTURE CLAIMS). SECOND TERM: ESTIMATION ERROR (UNCERTAINTY IN THE DEVELOPMENT FACTORS THEMSELVES).

For the total IBNR across all accident years, we must account for correlation between years (they share the same estimated development factors):

$$MSE(\text{Total IBNR}) = \sum_i MSE(\hat{C}_i) + 2 \times \sum_{i < k} \hat{C}(i,n) \times \hat{C}(k,n) \times \sum_j [\sigma^2(j)/\hat{f}^2(j)] \times [1/\sum_m C(m,j)]$$

THE CROSS-TERM CAPTURES THE POSITIVE CORRELATION BETWEEN AY RESERVES – A CRITICAL POINT OFTEN MISSED. TOTAL SE > SQRT(SUM OF AY SES SQUARED).

$$SE(\text{IBNR}) = \sqrt{MSE(\text{Total IBNR})}$$

STANDARD ERROR OF THE TOTAL RESERVE ESTIMATE. COV = SE/IBNR × 100%

Mack Implementation in Python

```

# =====
# Mack Chain Ladder – Full Implementation
# Reference: Mack (1993) "Distribution-Free Calculation of the
#           Standard Error of Chain Ladder Reserve Estimates"
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def mack_chain_ladder(paid_tri, tail=1.005):
    """
    Full Mack (1993) chain ladder with standard error derivation.

    Parameters:
        paid_tri: DataFrame – cumulative paid triangle (AY × Dev)
        tail: float – tail factor beyond last development period

    Returns:
        dict with ultimates, IBNR, SE by AY, and total portfolio SE
    """
    tri = paid_tri.values.copy().astype(float)
    n_ay, n_dev = tri.shape

    # — Step 1: Volume-Weighted Link Ratios —————
    f = np.ones(n_dev) # f[j] = development factor from period j to j+1
    for j in range(n_dev - 1):
        num = den = 0.0
        for i in range(n_ay):
            if not (np.isnan(tri[i,j]) or np.isnan(tri[i,j+1])):
                num += tri[i, j+1]
                den += tri[i, j]
        f[j] = num / den if den > 0 else 1.0
    f[n_dev - 1] = tail

    # — Step 2: CDF to Ultimate —————
    cdf = np.ones(n_dev)

```

```

cdf[n_dev - 1] = tail
for j in range(n_dev - 2, -1, -1):
    cdf[j] = cdf[j+1] * f[j]

# — Step 3: Find latest development per AY —————
latest_j = np.zeros(n_ay, dtype=int)
for i in range(n_ay):
    obs = np.where(~np.isnan(tri[i, :]))[0]
    latest_j[i] = obs.max() if len(obs) > 0 else 0

# — Step 4: Estimate  $\sigma^2$  parameters —————
# Mack Assumption M2:  $\text{Var}[C(i,j+1)|C(i,j)] = C(i,j) \times \sigma^2(j)$ 
sigma_sq = np.zeros(n_dev)
for j in range(n_dev - 1):
    ws = 0.0; cnt = 0
    W_j = 0.0 # sum of C(i,j) for pairs
    for i in range(n_ay):
        if not (np.isnan(tri[i,j]) or np.isnan(tri[i,j+1])):
            dev_factor_i = tri[i,j+1] / tri[i,j]
            ws += tri[i,j] * (dev_factor_i - f[j]) ** 2
            W_j += tri[i,j]
            cnt += 1

    # df = number of pairs minus 1 (one factor estimated)
    df = cnt - 1
    sigma_sq[j] = ws / df if df > 0 else 0

#  $\sigma^2(n-1)$ : extrapolate via Clark minimum of  $\{\sigma^2(n-2), \min(\sigma^2(n-3), \sigma^2(n-2))\}$ 
if n_dev >= 3:
    sigma_sq[n_dev-1] = min(sigma_sq[n_dev-2] ** 2 / sigma_sq[n_dev-3],
                             min(sigma_sq[n_dev-3], sigma_sq[n_dev-2]))

# — Step 5: Project ultimates —————
ultimates = np.zeros(n_ay)
latest_paid = np.zeros(n_ay)
for i in range(n_ay):
    latest_paid[i] = tri[i, latest_j[i]]
    ultimates[i] = latest_paid[i] * cdf[latest_j[i]]

```

```

ibnr = ultimates - latest_paid

# — Step 6: MSE by Accident Year (Mack 1993 Formula) —
#  $MSE_i = \hat{C}_i^2 \times \sum_{j=a(i)}^{n-1} [\sigma^2(j)/\hat{f}^2(j)] \times [1/C(i,j) + 1/\sum_k C(k,j)]$ 

# Precompute  $\sum_k C(k,j)$  for each development period j
sum_C_j = np.zeros(n_dev)
for j in range(n_dev):
    for i in range(n_ay):
        if not np.isnan(tri[i,j]) and (j < n_dev - 1):
            # Only periods where LDF is estimated from data
            if not np.isnan(tri[i, j+1] if j+1 < n_dev else np.nan):
                sum_C_j[j] += tri[i,j]

mse_ay = np.zeros(n_ay)
for i in range(n_ay):
    sum_term = 0.0
    for j in range(latest_j[i], n_dev - 1):
        # Individual policy at period j
        C_ij = tri[i, j] if j == latest_j[i] else (
            tri[i, latest_j[i]] * np.prod(f[latest_j[i]:j]))
        indiv_term = 1 / C_ij if C_ij > 0 else 0
        pool_term = 1 / sum_C_j[j] if sum_C_j[j] > 0 else 0
        sum_term += (sigma_sq[j] / f[j]**2) * (indiv_term + pool_term)
    mse_ay[i] = ultimates[i]**2 * sum_term

se_ay = np.sqrt(mse_ay)

# — Step 7: Total Portfolio MSE (includes cross-terms) —
# Total MSE ≠ sum of AY MSEs – add positive correlation cross-terms
mse_total = mse_ay.sum()

for i1 in range(n_ay):
    for i2 in range(i1 + 1, n_ay):
        cross = 0.0
        for j in range(max(latest_j[i1], latest_j[i2]), n_dev - 1):
            pool_term = 1 / sum_C_j[j] if sum_C_j[j] > 0 else 0
            cross += sigma_sq[j] / f[j]**2 * pool_term

```

```

        mse_total += 2 * ultimates[i1] * ultimates[i2] * cross

se_total = np.sqrt(max(mse_total, 0))
cov = se_total / ibnr.sum() if ibnr.sum() > 0 else np.nan

# — Step 8: Approximate 95% CI —————
# Mack uses lognormal for CI since reserves are positive
#  $\log(\text{Total IBNR}) \sim \text{Normal}(\mu_{\ln}, \sigma_{\ln}^2)$ 
sigma_ln_sq = np.log(1 + (se_total / ibnr.sum())**2)
mu_ln = np.log(ibnr.sum()) - sigma_ln_sq / 2
ci_lower = np.exp(mu_ln - 1.96 * np.sqrt(sigma_ln_sq))
ci_upper = np.exp(mu_ln + 1.96 * np.sqrt(sigma_ln_sq))

results_df = pd.DataFrame({
    'Latest Paid':    latest_paid.round(1),
    'CL Ultimate':    ultimates.round(1),
    'IBNR':           ibnr.round(1),
    'Mack SE':        se_ay.round(1),
    'CoV (%)':        (se_ay / np.where(ibnr>0, ibnr, 1) * 100).round(1),
    '95% CI Lower':   (ibnr - 1.96*se_ay).round(1),
    '95% CI Upper':   (ibnr + 1.96*se_ay).round(1),
}, index=paid_tri.index)

return {
    'by_ay':          results_df,
    'total_ibnr':      ibnr.sum(),
    'total_se':        se_total,
    'cov':             cov,
    'ci_lower':        ci_lower,
    'ci_upper':        ci_upper,
    'ldfs':           f,
    'sigma_sq':        sigma_sq,
}

mack = mack_chain_ladder(paid, tail=1.005)
print("=== MACK CHAIN LADDER RESULTS ===")
print(mack['by_ay'].to_string())
print(f"\nTotal IBNR: ₹{mack['total_ibnr']:,.1f} Lakhs")

```



```
print(f"Mack SE:      ₹{mack['total_se']:, .1f} Lakhs")
print(f"CoV:         {mack['cov']:.1%}")
print(f"95% CI:      ₹{mack['ci_lower']:, .1f} – ₹{mack['ci_upper']:, .1f} Lakhs")
```

Mack in R — ChainLadder Package

```
# =====
# Mack Chain Ladder in R (ChainLadder package)
# Reference: Gesmann et al., ChainLadder package
# =====

library(ChainLadder)
library(ggplot2)

# Load triangle (10x10 upper triangle)
paid_data <- read.csv("paid_triangle.csv", row.names = 1)

# Convert to triangle format
tri <- as.triangle(as.matrix(paid_data), 10, 10)

# — Mack method —————
mack_fit <- MackChainLadder(tri,
                             est.sigma = "Mack", # use Mack's sigma estimate
                             tail = 1.005,
                             alpha = 1,          # alpha=1: volume-weighted CL
                             mse.method = "Mack") # proper cross-term calculation

# — Summary output —————
print(mack_fit)
summary(mack_fit, "point")

# Key outputs:
# - IBNR by accident year
# - Standard error by accident year
# - CoV (= SE/IBNR)
# - Total IBNR with proper cross-year SE
```

```

# - 95% CI: [IBNR - 1.96*SE, IBNR + 1.96*SE] (normal approx)
# - Mack also produces upper 95% CI using lognormal

# — Mack diagnostic plots —————
plot(mack_fit, which=1:4) # 4 diagnostic plots:
# 1. Fitted vs standardised residuals (check M2 assumption)
# 2. Development years vs standardised residuals
# 3. Accident years vs standardised residuals
# 4. Calendar years vs standardised residuals (check M3)

# — Check Mack assumptions formally —————
MackChainLadder(tri, tail = 1.005)$Mack.S.E      # SE by AY
MackChainLadder(tri, tail = 1.005)$Mack.S.E[11]  # Total SE

# — Bootstrap ODP via ChainLadder —————
boot <- BootChainLadder(tri, R = 10000,
                        process.distr = "od.pois") # ODP process error
print(boot)
plot(boot) # Reserve distribution histogram

# Extract quantiles from bootstrap distribution
quantile(boot$IBNR, probs = c(0.5, 0.75, 0.90, 0.95, 0.995))

# — Bornhuetter-Ferguson in R —————
premium_vec <- c(5800, 6200, 6800, 7200, 7800,
                 8500, 9200, 9800, 10500, 11200)
apriori_lr  <- c(0.74, 0.75, 0.76, 0.75, 0.76,
                 0.77, 0.78, 0.78, 0.79, 0.80)

bf_fit <- BornhuetterFerguson(tri,
                              apriori = apriori_lr,
                              premium = premium_vec,
                              n.periods = 10)
print(bf_fit)

```

08

STOCHASTIC METHOD

Bootstrap Over-Dispersed Poisson (ODP) — Complete Theory & Code

ODP Bootstrap Framework

The Bootstrap ODP method (England & Verrall, 2002) is the most commonly used stochastic reserving technique in practice. Unlike the Mack model, it:

- Produces a **full simulated distribution** of the reserve, not just a mean and SE
- Can compute any risk measure directly from the distribution: VaR, TVaR, CoV
- Incorporates both **parameter uncertainty** (bootstrap resampling) and **process uncertainty** (ODP simulation)
- Is directly linked to the Poisson GLM chain ladder equivalence (England & Verrall 1999)

The Algorithm — Five Steps

1

Fit the Chain Ladder GLM and compute Pearson residuals

Fit the deterministic chain ladder; compute $\hat{m}(i,j)$ = expected incremental. Pearson residual: $r(i,j) = [c(i,j) - \hat{m}(i,j)] / \sqrt{\hat{m}(i,j)}$. Adjust for degrees of freedom.

2

Resample residuals with replacement (Bootstrap)

Sample $n \times (n-1)/2$ residuals with replacement from the adjusted residual set. Reconstruct bootstrap pseudo-triangle: $c^*(i,j) = \hat{m}(i,j) + r^*(i,j) \times \sqrt{\hat{m}(i,j)}$. This captures parameter uncertainty.

3

Refit the chain ladder on the pseudo-triangle

Compute new development factors $\hat{f}^*(j)$ from the bootstrap pseudo-triangle. Project to ultimate using these bootstrapped factors.

4

Add process error via ODP simulation

For each future cell, simulate incremental claims from ODP(mean = projected, dispersion = φ). Add simulated future cells to obtain one scenario total reserve. This captures process uncertainty (future randomness).

5

Repeat B = 10,000 times; extract distribution

Each iteration produces one total reserve scenario. The empirical distribution of 10,000 scenarios is the reserve distribution. Compute VaR, TVaR, CoV directly from the simulated reserve vector.

$$\hat{m}(i,j) = E[c(i,j)] = \hat{\alpha}_i \times \hat{\beta}_j \text{ [Fitted incremental (GLM form)]}$$

$$r(i,j) = [c(i,j) - \hat{m}(i,j)] / \sqrt{(\hat{\phi} \times \hat{m}(i,j))} \text{ [Scaled Pearson Residual]}$$

$$\hat{\phi} = \sum r^2(i,j) / (n(n-1)/2 - 2n+1) \text{ [ODP Dispersion Parameter]}$$

$\hat{\phi} > 1$ CONFIRMS OVERDISPERSION BEYOND POISSON. FOR MOTOR TP $\hat{\phi}$ IS TYPICALLY 150-500.

PYTHON

```
# =====
# Bootstrap ODP Reserving – Full Python Implementation
# Reference: England & Verrall (2002) "Stochastic Claims Reserving
#           in General Insurance"
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import warnings; warnings.filterwarnings('ignore')

np.random.seed(42)
TEAL='#1a8a7a'; AMBER='#d4882a'; INK='#0d0f14'

def odp_bootstrap_reserve(paid_tri_df, n_sim=10000, tail=1.005):
    """
    England & Verrall (2002) ODP Bootstrap.
    Produces full distribution of total outstanding reserves.

    Returns:
        reserve_dist: array of n_sim total reserve scenarios
        summary: dict with point estimates and risk measures
        phi: estimated ODP dispersion parameter
    """
    tri = paid_tri_df.values.copy().astype(float)
    n_ay, n_dev = tri.shape

    # — Step 1: Incremental triangle —————
    incr = np.zeros_like(tri)
```

```

incr[:, 0] = tri[:, 0]
for j in range(1, n_dev):
    incr[:, j] = tri[:, j] - tri[:, j-1]

# Mask future cells
mask_obs = np.zeros((n_ay, n_dev), dtype=bool)
for i in range(n_ay):
    for j in range(n_dev):
        if i + j < n_ay:
            mask_obs[i, j] = True

# — Step 2: Volume-weighted LDFs and cumulative factors —
f_fit = np.zeros(n_dev - 1)
for j in range(n_dev - 1):
    num = den = 0
    for i in range(n_ay):
        if mask_obs[i, j] and (j+1 < n_dev) and mask_obs[i, j+1]:
            num += tri[i, j+1]; den += tri[i, j]
    f_fit[j] = num/den if den>0 else 1.0

# CDF to ultimate
cdf_fit = np.ones(n_dev)
cdf_fit[-1] = tail
for j in range(n_dev-2, -1, -1):
    cdf_fit[j] = cdf_fit[j+1] * (f_fit[j] if j < len(f_fit) else tail)

# — Step 3: Fitted incrementals  $\hat{m}(i, j)$  —————
# Under the GLM/CL equivalence:  $\hat{m}(i, j) = \alpha_i \times \theta_j$ 
# Simpler:  $\hat{m}(i, j) = C(i, j-1) \times f(j-1) - C(i, j-1) = C(i, j-1) \times (f(j-1)-1)$ 
m_hat = np.zeros_like(incr)
m_hat[:, 0] = incr[:, 0] # first col: observed
for j in range(1, n_dev):
    for i in range(n_ay):
        if mask_obs[i, j] and tri[i, j-1] > 0:
            m_hat[i, j] = tri[i, j-1] * (f_fit[j-1] - 1) if j-1<len(f_fit) else 0

# — Step 4: Pearson residuals —————
#  $r(i, j) = (c_{obs} - m_{hat}) / \text{sqrt}(m_{hat})$  [unscaled]

```

```

residuals = []
for i in range(n_ay):
    for j in range(1, n_dev): # skip j=0 (no development factor)
        if mask_obs[i,j] and m_hat[i,j] > 0:
            r = (incr[i,j] - m_hat[i,j]) / np.sqrt(m_hat[i,j])
            residuals.append((i, j, r))

resid_vals = np.array([r[2] for r in residuals])

# — Step 5: ODP Dispersion parameter  $\hat{\phi}$  —————
n_obs = len(residuals)
# df = n_obs - (n_ay-1) - (n_dev-1) - parameters in GLM
df_resid = n_obs - (n_ay - 1) - (n_dev - 1)
phi_hat = np.sum(resid_vals**2) / df_resid
print(f"ODP Dispersion parameter  $\hat{\phi}$  = {phi_hat:.1f}")
print(f"( $\hat{\phi}$  > 1 confirms overdispersion beyond Poisson)")

# Adjust residuals for df (standardise)
# Adjusted residual = r * sqrt(n_obs / df_resid)
adj_factor = np.sqrt(n_obs / df_resid)
resid_adj = resid_vals * adj_factor
# Centre residuals (subtract mean)
resid_adj -= resid_adj.mean()

# — Step 6: Find latest observed diagonal —————
latest_j = np.zeros(n_ay, dtype=int)
for i in range(n_ay):
    obs = np.where(mask_obs[i, :])[0]
    latest_j[i] = obs.max() if len(obs) > 0 else 0

latest_paid = np.array([tri[i, latest_j[i]] for i in range(n_ay)])

# — Step 7: Bootstrap Simulation Loop —————
reserve_dist = np.zeros(n_sim)

for b in range(n_sim):
    # — 7a: Resample residuals → pseudo incremental triangle
    resampled_r = np.random.choice(resid_adj, size=len(residuals), replace=True)

```

```

pseudo_incr = incr.copy()
for idx, (i, j, _) in enumerate(residuals):
    if j > 0:
        pseudo_incr[i,j] = m_hat[i,j] + resampled_r[idx] * np.sqrt(m_hat[i,j])
        pseudo_incr[i,j] = max(pseudo_incr[i,j], 0.01) # floor at 0

# — 7b: Rebuild pseudo-cumulative triangle
pseudo_cum = np.zeros((n_ay, n_dev))
pseudo_cum[:, 0] = pseudo_incr[:, 0]
for j in range(1, n_dev):
    pseudo_cum[:, j] = pseudo_cum[:, j-1] + pseudo_incr[:, j]

    # Only populate observed cells
    for i in range(n_ay):
        if not mask_obs[i,j]:
            pseudo_cum[i, j] = np.nan

# — 7c: Refit CL on pseudo-triangle
f_boot = np.ones(n_dev - 1)
for j in range(n_dev - 1):
    num = den = 0
    for i in range(n_ay):
        if mask_obs[i,j] and mask_obs[i,j+1]:
            num += pseudo_cum[i,j+1]; den += pseudo_cum[i,j]
    f_boot[j] = num/den if den > 0 else f_fit[j]

# — 7d: Project future cells + add process error (ODP noise)
total_ibnr = 0.0
for i in range(n_ay):
    current = tri[i, latest_j[i]]
    for j in range(latest_j[i] + 1, n_dev):
        # Expected incremental from bootstrap CL
        idx_f = min(j-1, len(f_boot)-1)
        mean_incr = current * (f_boot[idx_f] - 1)
        mean_incr = max(mean_incr, 0.1)

        # Process error: ODP → simulate via Gamma(mean/φ, φ)
        sim_incr = np.random.gamma(mean_incr / phi_hat, phi_hat)
        current += sim_incr

```

```

    # Tail factor
    mean_tail = current * (tail - 1)
    sim_tail = np.random.gamma(max(mean_tail, 0.1)/phi_hat, phi_hat)
    ultimate_b = current + sim_tail
    total_ibnr += ultimate_b - latest_paid[i]

    reserve_dist[b] = total_ibnr

# — Step 8: Risk Measures from Distribution —————
reserve_dist = np.sort(reserve_dist)
be = np.mean(reserve_dist)

summary = {
    'Best Estimate (Mean)': np.round(be, 1),
    'Median (50th pct)': np.round(np.percentile(reserve_dist, 50), 1),
    'VaR(75%)': np.round(np.percentile(reserve_dist, 75), 1),
    'VaR(90%)': np.round(np.percentile(reserve_dist, 90), 1),
    'VaR(95%)': np.round(np.percentile(reserve_dist, 95), 1),
    'VaR(99.5%)': np.round(np.percentile(reserve_dist, 99.5), 1),
    'TVaR(99.5%)': np.round(np.mean(reserve_dist[reserve_dist >= np.percentile(reserve_dist, 99.5)]), 1),
    'Standard Deviation': np.round(np.std(reserve_dist), 1),
    'CoV (%)': np.round(np.std(reserve_dist)/be*100, 1),
    'Skewness': np.round(stats.skew(reserve_dist), 3),
    'Kurtosis': np.round(stats.kurtosis(reserve_dist), 3),
    'Buffer (VaR99.5 - BE)': np.round(np.percentile(reserve_dist, 99.5) - be, 1),
}

return reserve_dist, summary, phi_hat

# — Run Bootstrap —————
paid = pd.read_csv('paid_triangle.csv', index_col=0)
reserve_dist, summary, phi = odp_bootstrap_reserve(paid, n_sim=10000)

print("\n=== ODP BOOTSTRAP RESULTS ===")
for k, v in summary.items():
    print(f" {k:<30} ₹{v:,.1f} Lakhs" if isinstance(v, float) and v > 10 else f" {k:} {v}")

```



```
np.save('reserve_distribution.npy', reserve_dist)

print("\nReserve distribution saved to reserve_distribution.npy")
```

09

RISK ANALYSIS

Reserve Distribution & Risk Measures

PYTHON

```
# =====
# Reserve Distribution Analysis – VaR, TVaR, CoV, IFRS 17
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from scipy import stats

reserve_dist = np.load('reserve_distribution.npy')
TEAL='#1a8a7a'; AMBER='#d4882a'; INK='#0d0f14'; RED='#b53030'

# — 1. RESERVE DISTRIBUTION PLOT —————
def plot_reserve_distribution(dist, title='Reserve Distribution – ODP Bootstrap'):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle(title, fontsize=13, fontweight='bold', color=INK)

    be = np.mean(dist)
    p75 = np.percentile(dist, 75)
    p90 = np.percentile(dist, 90)
    p99 = np.percentile(dist, 99.5)

    # Histogram with shaded tails
    ax = axes[0]
    counts, bins, _ = ax.hist(dist, bins=100, density=True,
                               color=TEAL, alpha=0.6, edgecolor='white', linewidth=0.3
```

```

# Shade 75-99.5th percentile region
ax.axvline(be, color=INK, linewidth=2, label=f'Best Estimate ₹{be:,.0f}')
ax.axvline(p75, color='#4caf8a', linewidth=1.5, linestyle='--', label=f'VaR(75%) ₹{p75:,.0f}')
ax.axvline(p90, color=AMBER, linewidth=1.5, linestyle='--', label=f'VaR(90%) ₹{p90:,.0f}')
ax.axvline(p99, color=RED, linewidth=2, linestyle='-', label=f'VaR(99.5%) ₹{p99:,.0f}')

# Shade tail region
tail_mask = dist >= p99
ax.fill_betweenx([0, counts.max()], p99, dist.max(),
                 alpha=0.2, color=RED, label='99.5th+ tail')

ax.set_xlabel('Total IBNR Reserve (₹ Lakhs)')
ax.set_ylabel('Probability Density')
ax.set_title('Reserve Distribution Histogram')
ax.legend(fontsize=8)

# Empirical CDF
ax2 = axes[1]
sorted_d = np.sort(dist)
cdf = np.arange(1, len(sorted_d)+1) / len(sorted_d)
ax2.plot(sorted_d, cdf, color=TEAL, linewidth=2)
ax2.axhline(0.75, color='#4caf8a', linestyle='--', alpha=0.7, label='75th %ile')
ax2.axhline(0.90, color=AMBER, linestyle='--', alpha=0.7, label='90th %ile')
ax2.axhline(0.995, color=RED, linestyle='-', alpha=0.9, label='99.5th %ile')
ax2.axvline(be, color=INK, linewidth=1.5, linestyle=':', label='Best Estimate')
ax2.set_xlabel('Total IBNR Reserve (₹ Lakhs)')
ax2.set_ylabel('Cumulative Probability')
ax2.set_title('Empirical CDF')
ax2.legend(fontsize=8)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('reserve_distribution.png', dpi=150, bbox_inches='tight')
return fig

plot_reserve_distribution(reserve_dist)

```

— 2. RISK MEASURES SUMMARY

```

be = np.mean(reserve_dist)
med = np.median(reserve_dist)
sd = np.std(reserve_dist)
cov = sd / be

risk_df = pd.DataFrame({
    'Percentile': ['Mean (BE)', 'Median', '75th', '90th', '95th',
                  '99th', '99.5th', 'TVaR(99.5%)'],
    'Reserve (₹ Lakhs)': [
        be, med,
        np.percentile(reserve_dist, 75),
        np.percentile(reserve_dist, 90),
        np.percentile(reserve_dist, 95),
        np.percentile(reserve_dist, 99),
        np.percentile(reserve_dist, 99.5),
        np.mean(reserve_dist[reserve_dist >= np.percentile(reserve_dist, 99.5)])
    ],
    'Buffer above BE': [0, med-be,
        np.percentile(reserve_dist, 75)-be,
        np.percentile(reserve_dist, 90)-be,
        np.percentile(reserve_dist, 95)-be,
        np.percentile(reserve_dist, 99)-be,
        np.percentile(reserve_dist, 99.5)-be,
        np.mean(reserve_dist[reserve_dist >= np.percentile(reserve_dist, 99.5)])-be
    ],
})

risk_df['Reserve (₹ Lakhs)'] = risk_df['Reserve (₹ Lakhs)'].round(1)
risk_df['Buffer above BE'] = risk_df['Buffer above BE'].round(1)
print("=== RESERVE RISK MEASURES ===")
print(risk_df.to_string(index=False))
print(f"\nCoV = {cov:.1%} | Skewness = {stats.skew(reserve_dist):.3f}")

```

— 3. NORMALITY CHECK

```

# Is the distribution approximately lognormal?
# Many actuarial frameworks assume lognormal for reserves

log_dist = np.log(reserve_dist[reserve_dist > 0])
_, p_norm = stats.normaltest(log_dist)

```

```

print(f"\nLog-normality test p-value: {p_norm:.3f}")
print(f"'Cannot reject lognormal (p>0.05)' if p_norm>0.05 else 'Lognormal not well-su

# — 4. IFRS 17 RISK ADJUSTMENT —————
# Risk Adjustment = the amount above BE that makes the insurer
# indifferent between bearing the risk and eliminating it
# Most insurers set RA = amount to reach 75th-85th percentile confidence

print("\n=== IFRS 17 RISK ADJUSTMENT ANALYSIS ===")
for conf_level in [0.75, 0.80, 0.85, 0.90]:
    var = np.percentile(reserve_dist, conf_level * 100)
    ra = var - be
    ra_pct = ra / be * 100
    print(f"  RA at {conf_level:.0%} CL: ₹{ra:,.1f} Lakhs ({ra_pct:.1f}% of BE)")

# — 5. RESERVE ADEQUACY ANALYSIS —————
CARRIED_RESERVE = 42000 # ₹42,000 Lakhs = ₹420 Cr (management carried reserve)
pct_adequate = np.mean(reserve_dist <= CARRIED_RESERVE) * 100
shortfall_scenarios = np.sum(reserve_dist > CARRIED_RESERVE)
expected_shortfall = np.mean(np.maximum(reserve_dist - CARRIED_RESERVE, 0))

print(f"\n=== RESERVE ADEQUACY ===")
print(f"  Carried Reserve:      ₹{CARRIED_RESERVE:,} Lakhs")
print(f"  Best Estimate (Mean):   ₹{be:,.1f} Lakhs")
print(f"  Confidence Level:      {pct_adequate:.1f}th percentile")
print(f"  Scenarios with deficit: {shortfall_scenarios:,} / {len(reserve_dist):,}")
print(f"  Expected Shortfall:     ₹{expected_shortfall:,.1f} Lakhs")

```

Risk Measure Definitions — Interview Essentials

MEASURE	DEFINITION	REGULATORY USE
Best Estimate (Mean)	$E[R]$ — expected value of the reserve distribution	IFRS 17 BEL; IRDAI IBNR best estimate
Median	50th percentile — for skewed distributions, median > mean	Management comparison point; mean > median signals positive skew

MEASURE	DEFINITION	REGULATORY USE
VaR(p)	Value at Risk at confidence $p = \inf\{r: P(R \leq r) \geq p\}$	VaR(75%) = IFRS 17 RA benchmark; VaR(99.5%) = Solvency II SCR
TVaR(p)	Tail VaR = $E[R \mid R \geq \text{VaR}(p)]$ — mean of the tail beyond VaR	Better than VaR for capital: accounts for tail shape, not just threshold
CoV	Coefficient of Variation = $SD(R) / E[R]$ — relative uncertainty	CoV <10% = low uncertainty; 10–20% = moderate; >20% = high
Risk Adjustment (IFRS 17)	RA = VaR(confidence level) – Best Estimate	Must disclose the confidence level corresponding to the RA

10

DIAGNOSTICS

Model Validation & Diagnostics

PYTHON

```
# =====  
  
# Reserving Model Validation – Full Diagnostic Framework  
  
# =====  
  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# — 1. RESIDUAL ANALYSIS —————  
# Mack Assumption M1 test: residuals should have no pattern  
# against fitted values, development period, accident year, calendar year  
  
def compute_standardised_residuals(paid_tri):  
    """Standardised Mack residuals for assumption testing"""  
    tri = paid_tri.values.copy().astype(float)  
    n_ay, n_dev = tri.shape  
  
    residuals = []  
    for i in range(n_ay):  
        for j in range(1, n_dev):
```

```

    if i + j < n_ay: # observed cell
        if tri[i,j-1] > 0 and not np.isnan(tri[i,j]):
            f_j = ldf[j-1] # from Mack results
            fitted = tri[i,j-1] * f_j
            raw_r = tri[i,j] - fitted
            # Standardise by sqrt(fitted) - Mack M2
            std_r = raw_r / np.sqrt(fitted) if fitted > 0 else 0
            residuals.append({
                'AY': paid_tri.index[i],
                'Dev': j + 1,
                'CY': paid_tri.index[i] + j, # calendar year
                'fitted': fitted,
                'std_resid': std_r
            })

    return pd.DataFrame(residuals)

paid = pd.read_csv('paid_triangle.csv', index_col=0)
ldf = mack['ldfs'][:-1] # exclude tail
resid_df = compute_standardised_residuals(paid)

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Mack Residual Diagnostics (Test Assumptions M1-M3)',
             fontsize=12, fontweight='bold')

# Residuals vs Fitted
axes[0,0].scatter(np.log(resid_df['fitted']+0.01), resid_df['std_resid'],
                  alpha=0.6, color='#1a8a7a', s=20)
axes[0,0].axhline(0, color='#d4882a', linewidth=1.5, linestyle='--')
axes[0,0].set_xlabel('log(Fitted)'); axes[0,0].set_ylabel('Std Residual')
axes[0,0].set_title('vs Fitted Values (M1 test: no pattern expected)')

# Residuals vs Development Period
axes[0,1].scatter(resid_df['Dev'], resid_df['std_resid'],
                  alpha=0.6, color='#1a8a7a', s=20)
axes[0,1].axhline(0, color='#d4882a', linewidth=1.5, linestyle='--')
axes[0,1].set_xlabel('Development Period'); axes[0,1].set_ylabel('Std Residual')
axes[0,1].set_title('vs Development Period (M2 test: no trend expected)')

```

```

# Residuals vs Accident Year
axes[1,0].scatter(resid_df['AY'], resid_df['std_resid'],
                  alpha=0.6, color='#1a8a7a', s=20)
axes[1,0].axhline(0, color='#d4882a', linewidth=1.5, linestyle='--')
axes[1,0].set_xlabel('Accident Year'); axes[1,0].set_ylabel('Std Residual')
axes[1,0].set_title('vs Accident Year (M3 test: independence)')

# Residuals vs Calendar Year (diagonal test)
axes[1,1].scatter(resid_df['CY'], resid_df['std_resid'],
                  alpha=0.6, color='#1a8a7a', s=20)
axes[1,1].axhline(0, color='#d4882a', linewidth=1.5, linestyle='--')
axes[1,1].set_xlabel('Calendar Year'); axes[1,1].set_ylabel('Std Residual')
axes[1,1].set_title('vs Calendar Year (CY inflation check)')

plt.tight_layout()
plt.savefig('mack_residuals.png', dpi=150, bbox_inches='tight')

# — 2. BACK-TESTING: HINDCAST ACCURACY —————
# Remove one year from triangle, re-estimate, compare vs actual
# Tests whether the model would have predicted actual development correctly
def hindcast_test(paid_tri, ldf_func):
    """
    Exclude latest diagonal, re-estimate CL, compare prediction vs actuality.
    Key metric: actual / predicted (should be close to 1.0)
    """
    tri = paid_tri.values.copy().astype(float)
    n_ay = tri.shape[0]
    results = []

    for i in range(2, n_ay):
        # Observed at period i-1
        obs_prev = tri[i, n_ay-i-1]
        # Actual at period i (latest)
        obs_curr = tri[i, n_ay-i]
        if not (np.isnan(obs_prev) or np.isnan(obs_curr)):
            predicted = obs_prev * ldf[n_ay-i-1]
            results.append({
                'AY': paid_tri.index[i],

```

```

        'Actual': obs_curr,
        'Predicted': predicted,
        'AvP Ratio': obs_curr / predicted,
        'Error %': (obs_curr - predicted) / predicted * 100
    })

    return pd.DataFrame(results)

hindcast = hindcast_test(paid, ldf)
print("\n=== HINDCAST BACK-TEST ===")
print(hindcast.round(3).to_string())
print(f"\nMean AvP Ratio: {hindcast['AvP Ratio'].mean():.3f}")
print(f"Std Dev AvP:      {hindcast['AvP Ratio'].std():.3f}")
print("Target: mean ≈ 1.00, SD < 0.10 for a well-calibrated model")

# — 3. TAIL FACTOR SENSITIVITY —————
print("\n=== TAIL FACTOR SENSITIVITY ===")
for tail in [1.000, 1.003, 1.005, 1.010, 1.020]:
    _, s, _ = mack_chain_ladder(paid, tail=tail).values()
    total_ibnr = s['total_ibnr'] if isinstance(s, dict) else s
    print(f"Tail = {tail:.3f}: IBNR ≈ ₹{mack_chain_ladder(paid, tail=tail)['total_ibnr']:.2f}")

```

Diagnostic Interpretation Guide

PLOT	WHAT PASSES	WHAT FAILS & ACTION
Residuals vs Fitted	Random scatter around 0; no fan/funnel shape	Fan shape → Variance not proportional to $C(i,j)$; try log-normal instead of Mack
Residuals vs Dev Period	No trend across development periods	Positive trend in early periods → LDF systematically underestimates early development; select different factor methods
Residuals vs AY	All AYs randomly scatter; no one year dominates	Positive cluster for 2017 → excluded/down-weighted AY; consider excluding from LDF averaging
Residuals vs CY (diagonal)	No pattern across calendar diagonals	Positive 2020–2022 → COVID claims inflation; apply diagonal factor or Clark CPLDF method

PLOT	WHAT PASSES	WHAT FAILS & ACTION
Hindcast AvP ratio	Mean $\approx$ 1.0; SD < 0.10	Mean > 1.05 $\rightarrow$ CL systematically underreserves; add explicit IBNER loading

11

RECOMMENDATIONS

Business Interpretation

Summary of Results — All Methods

METHOD	TOTAL IBNR ₹ LAKHS	SE / UNCERTAINTY	CONFIDENCE LEVEL
Chain Ladder (deterministic)	41,850	— (point estimate)	50th pctile approximately
Bornhuetter–Ferguson	40,200	— (point estimate)	50th pctile approximately
Mack Chain Ladder	41,850	SE = 4,220 (CoV 10.1%)	95% CI: ₹33,570–₹50,130
Bootstrap ODP (mean)	42,180	SD = 4,850 (CoV 11.5%)	Full distribution available
Carried Reserve (Management)	42,000	—	~52nd percentile (Bootstrap)

Risk Measure Summary for Stakeholders

RISK MEASURE	RESERVE ₹ LAKHS	BUFFER ABOVE BE	USE
Best Estimate	42,180	—	IFRS 17 BEL; reported IBNR
VaR(75%)	45,250	+3,070	IFRS 17 Risk Adjustment benchmark
VaR(90%)	48,400	+6,220	Management buffer requirement
VaR(99.5%)	56,800	+14,620	Solvency II / IRDAI RBC capital
TVaR(99.5%)	59,200	+17,020	Economic capital; reinsurance sizing

Key Findings for Management

FINDING 1: CARRIED RESERVE IS ADEQUATE AT 52ND PERCENTILE

The carried reserve of ₹42,000 Lakhs sits approximately at the 52nd percentile of the bootstrap distribution. This means the reserve is adequate under 52% of scenarios — **broadly consistent with a best estimate reserve** as required by IRDAI and IFRS 17. The reserve is not being held at a material margin above best estimate, which is expected for a GI insurer carrying reserves at the gross liability level.

FINDING 2: 2017 ACCIDENT YEAR IS THE KEY UNCERTAINTY DRIVER

The AY 2017 IBNR has a CoV of 22% — more than double the portfolio average — driven by the court ruling shock. This year accounts for 38% of the total portfolio SE despite representing only 11% of the IBNR. Management should consider additional case review for AY 2017 claims and consult with the Legal team on outstanding court cases.

FINDING 3: IFRS 17 RISK ADJUSTMENT

To set the Risk Adjustment (RA) at a 75% confidence level as commonly used in the industry, the insurer would hold an additional ₹3,070 Lakhs above the best estimate. This represents 7.3% of the BEL. The RA must be disclosed in the IFRS 17 financial notes along with the confidence level used.

FINDING 4: CAPITAL REQUIREMENT (VAR 99.5%) IS ₹14,620 LAKHS

The reserve risk capital at the 99.5th percentile (Solvency II / IRDAI RBC standard) is ₹14,620 Lakhs above the best estimate. This is the reserve risk contribution to the Solvency Capital Requirement. Management should verify this is consistent with the Standard Formula reserve risk charge and consider internal model approval if materially different.

Recommended Actions

1

Maintain current carried reserve at best estimate level

The 52nd percentile position is appropriate for a gross best estimate reserve. No immediate strengthening required unless BF or alternative methods indicate a systematic shortfall.

2

Strengthen AY 2017 case reserve review

- 3

Set IFRS 17 Risk Adjustment at 75th percentile (₹3,070 Lakhs)

This is competitive with peer insurers. Revisit annually as the portfolio composition changes. Consider calibrating against a Cost of Capital approach as an alternative.
- 4

Quarterly monitoring of emerging development

Implement AvE monitoring dashboard (Section 12) to detect reserve drift. Alert threshold: actual claims > expected by >10% for any accident year in any quarter.

12

DASHBOARD

Power BI Dashboard Design

Page 1 — Reserving Executive Summary



Page 2 — Reserve Distribution (Stochastic)

PAGE 2: ODP BOOTSTRAP RESERVE DISTRIBUTION – 10,000 SCENARIOS

 Histogram: Bootstrap Reserve Distribution
Shaded regions: 50-75th, 75-90th, 90-99.5th, 99.5th+
Vertical lines: BE, VaR75, VaR99.5, Carried Reserve

 Line: Empirical CDF
X-axis: reserve amount | Y-axis: $P(\text{IBNR} \leq X)$
Dotted horizontal: 75%, 90%, 95%, 99.5%

 Table: Risk Measures
BE | VaR75 | VaR90 | VaR99.5 | TVaR | CoV

 Bar: CoV by AY
Relative uncertainty – AY 2017 flagged

 IFRS 17 RA Panel
RA at 75%, 80%, 85% CL + chosen level

Page 3 — Development Triangle Monitoring

PAGE 3: CLAIMS DEVELOPMENT MONITORING – ACTUAL VS EXPECTED

 Matrix/Heatmap: Paid Triangle
Colour = AvE ratio (Actual / CL Expected)
Red > 110%; Amber 100-110%; Green < 100%

 Line Chart: LDF Stability by Dev Period
Individual LDFs (dots) vs VWA (line) vs Prior period VWA
Spot changes in development speed

 Paid LR Development Lines by Accident Year
One line per AY; latest diagonal highlighted; divergence = concern

Key DAX Measures

DAX

```
-- Best Estimate IBNR (sum across AYs from CL model)
Best Estimate IBNR =
    SUM(Reserves[CL_IBNR])

-- AvE Ratio – Actual paid vs Expected (CL projection at latest diagonal)
AvE Ratio =
    DIVIDE(SUM(Triangle[Actual_Paid]), SUM(Triangle[CL_Expected]), 1)

-- CoV from Bootstrap results
CoV =
    DIVIDE([Bootstrap_SD], [Bootstrap_Mean], 0)
```

```

-- Reserve Development Quarter on Quarter
IBNR QoQ Change =
    [Best Estimate IBNR] -
    CALCULATE([Best Estimate IBNR], PREVIOUSQUARTER(Calendar[Date]))

-- Development Factor (LDF) – computed from triangle data
LDF =
    DIVIDE(
        SUMX(Triangle, Triangle[Cum_j1] * NOT(ISBLANK(Triangle[Cum_j]))),
        SUMX(Triangle, Triangle[Cum_j] * NOT(ISBLANK(Triangle[Cum_j1]))),
        1
    )

-- Confidence Level – percentile of carried vs bootstrap distribution
-- Computed outside DAX via Python, stored as a measure
Reserve Confidence Level =
    VAR CarriedReserve = [Carried_Reserve]
    RETURN
    COUNTROWS(FILTER(Bootstrap_Scenarios, Bootstrap_Scenarios[IBNR] <= CarriedReserve))
    / COUNTROWS(Bootstrap_Scenarios)

```

13

CAREER PREP

80 Interview Questions & Ideal Answers

A. Reserving Fundamentals (20 Questions)

q What is the difference between IBNR and IBNER? Why does it matter for reserving?

notified to the insurer — pure emergence risk. IBNER (Incurred But Not Enough Reserved) relates to reported claims where the case reserves are inadequate — reserve development risk. The distinction matters because: (1) IBNR is harder to estimate — it requires predicting the volume of future notifications; (2) IBNER can be partially addressed by large claims reviews; (3) Different lines have different IBNR/IBNER mixes — short-tail property is mostly IBNR-light (claims reported quickly), while long-tail TP motor liability

has significant IBNR components. Under IFRS 17, both contribute to the Liability for Incurred Claims, but the Risk Adjustment may be calibrated differently for IBNR vs IBNER if the uncertainty profiles differ materially.

q Explain the three Mack assumptions. How would you test each one?

Ideal Answer: M1: $E[C(i,j+1)|C(i,j)] = C(i,j) \times f(j)$ — the expected development depends only on the latest cumulative (Markov property) and $f(j)$ is the same for all AYs. Test: plot standardised residuals against fitted values; no pattern expected. M2: $\text{Var}[C(i,j+1)|C(i,j)] = C(i,j) \times \sigma^2(j)$ — variance proportional to current cumulative. Test: plot residuals against development period; no heteroscedasticity. M3: Accident years are independent — shared development factor estimates are the only connection. Test: plot residuals against calendar year diagonals — systematic pattern would indicate calendar year effects (claims inflation, settlement changes) violating independence. The Venter (1998) test formally checks M1 using an F-statistic; the Mack (1994) calendar year test formally tests M3.

q When would you use BF instead of Chain Ladder? When would you blend them?

Ideal Answer: BF is preferred when: (1) AYs are immature (less than 30–40% developed) — CL amplifies any process noise dramatically; (2) the a priori expected loss ratio is reliable (set by an experienced pricing actuary); (3) there are negative incremental cells or very high individual LDFs — BF is more stable; (4) the line of business is new (insufficient historical data). CL is preferred when: (1) AYs are mature (>60%); (2) the historical development pattern is highly stable; (3) there's no reliable a priori. Blending: the Benktander/Gunnar method takes one iteration — use BF IBNR as the new a priori for a second BF pass. The Bornhuetter (1972) credibility argument: weight CL and BF by the development factor, with more credibility to CL as the year matures.

q Why is the tail factor important? How would you estimate it for a long-tail line?

Ideal Answer: The tail factor captures development beyond the last observed development period. For motor TP liability with 10 development years, there may still be 0.5–2% of claims outstanding beyond year 10 (court cases, disability pension claims). Underestimating the tail systematically underestimates IBNR. Estimation methods: (1) Curve fitting — fit a parametric curve (inverse power, exponential) to the observed LDFs and extrapolate beyond the data; (2) Industry benchmarks — use market data from Lloyd's or reinsurers for your line; (3) Inverse power: $f(j) = 1 + a/j^b$ — extrapolate to sum the tail; (4) The Sherman (1984) method: regress $\log(f(j)-1)$ on $\log(j)$ and extrapolate. The tail factor is often the largest single source of estimation uncertainty in long-tail lines — a 1% tail factor error on a long-tail book can equate to 5–10% IBNR error.

q What is the ODP dispersion parameter ϕ and why does it matter for bootstrapping?

Ideal Answer: The Over-Dispersed Poisson dispersion parameter φ captures the degree of randomness beyond what a standard Poisson distribution would predict. For standard Poisson: $\text{Var}[Y] = E[Y]$. For ODP: $\text{Var}[Y] = \varphi \times E[Y]$. φ is estimated from the Pearson residuals: $\hat{\varphi} = \sum r^2(i,j) / (n_{\text{obs}} - \text{degrees_of_freedom})$. For motor TP, φ is typically 150–500 — highly overdispersed relative to Poisson. Why it matters: in the bootstrap, φ scales the process error simulation (future incremental claims are drawn from $\text{ODP}(\text{mean}, \varphi) = \text{Gamma}(\text{mean}/\varphi, \varphi)$). A higher φ widens the reserve distribution — larger SE, higher VaR. If φ is estimated poorly (e.g., from too few cells), the entire reserve distribution is miscalibrated. Note: $\varphi > 1$ confirms the data is overdispersed — if $\varphi \approx 1$, the standard Poisson model is adequate and bootstrapping reduces to simpler parametric methods.

q What is the Coefficient of Variation (CoV) for reserve estimates? What is a typical CoV for motor TP?

Ideal Answer: $\text{CoV} = \text{Standard Deviation} / \text{Mean}$ — the relative uncertainty of the reserve estimate. CoV captures model and process uncertainty in a single number useful for management communication. Typical ranges: short-tail property: 5–10%; motor OD: 8–15%; motor TP liability: 10–20%; employers' liability: 20–40%; casualty/PPO reserves: 40%+. A CoV >25% should prompt management questions about why uncertainty is high — is the triangle immature? Are there large case reserves for complex claims? Are development patterns unstable? IRDAI Appointed Actuaries are expected to comment on material reserve uncertainty in the Actuarial Report. Under IFRS 17, the Risk Adjustment is directionally related to the CoV — higher CoV implies higher RA.

q How does the bootstrap ODP account for both parameter uncertainty and process uncertainty?

Ideal Answer: These are two distinct sources of reserve uncertainty: (1) Parameter uncertainty: we don't know the true development factors — they must be estimated from data. Bootstrap addresses this by resampling Pearson residuals to create pseudo-triangles, then re-estimating the development factors on each pseudo-triangle. Each iteration uses slightly different $\hat{f}(j)$ — this variation across simulations is the parameter uncertainty component. (2) Process uncertainty: even if we knew the exact development factors, future claims still have random variability. The ODP simulation step addresses this — for each future cell, we draw from an $\text{ODP}(\text{mean} = \text{projected}, \varphi)$ distribution rather than using the point estimate. The total bootstrap variance = parameter variance + process variance. An important check: if you remove the process error step, the distribution narrows but doesn't collapse — that remaining width is pure parameter uncertainty. Mack's formula primarily captures parameter uncertainty; the bootstrap adds process uncertainty, which is why bootstrap SE is usually slightly larger than Mack SE.

q What is a volume-weighted average link ratio? Why not use a simple average?

Ideal Answer: The volume-weighted average LDF weights each observed pair $(C(i,j), C(i,j+1))$ by the denominator $C(i,j)$: $\hat{f}(j) = \sum C(i,j+1) / \sum C(i,j)$. This gives larger, more mature accident years more influence. Simple average would give a small AY (e.g., 2023 with £2m exposure) the same weight as a large one (2014 with £10m). Under Mack's M2 assumption — $\text{Var} \propto C(i,j)$ — the volume-weighted estimator is the minimum

variance unbiased estimator of $f(j)$. A simple average assumes each individual LDF has equal variance — inconsistent with M2. In practice, you compute both and compare; if they differ materially, investigate which AYs are pulling the simple average away from the VWA. Other options: 3-year VWA (recent data only), 5-year VWA, medial (drop high/low), all-year VWA. Selection requires actuarial judgment.

q What are the limitations of the chain ladder method?

Ideal Answer: Six key limitations: (1) Assumes stable development pattern — structural changes (court awards, settlement speed, claims handling system) break this assumption; (2) Amplifies accident year noise — immature years with high early paid get projected forward at a high multiple; (3) No mechanism for a priori expectation — purely data-driven; new or unusual years are handled poorly; (4) Calendar year effects — inflation across diagonals violates the independence assumption M3; (5) Large single claims — one large claim in an early cell dramatically changes the LDF for that development period; (6) Zero/negative incrementals — can't handle cells with no claims or reserve releases well; produces anomalous projections. Mitigations: BF for immature years; explicit large claim separation; Clark curve fitting for parametric stability; GLM reserving (Section 15) for explicit factor modelling.

q Describe the Mack cross-term in the total portfolio MSE. Why is it important?

Ideal Answer: The Mack total portfolio SE includes cross-terms that capture the positive correlation between accident year reserves — they all share the same estimated development factors. The formula is: $MSE(\text{total}) = \sum MSE(i) + 2 \times \sum_{i < k} \hat{C}_i \times \hat{C}_k \times \sum_j [\sigma^2(j)/\hat{f}^2(j)] / \sum C(m_j)$. The cross-term is always positive — total SE > $\sqrt{\sum MSE(i)}$ — meaning you cannot calculate portfolio SE by adding AY SEs in quadrature. Why important: (1) Management needs the total portfolio SE for capital purposes; (2) Using summed-in-quadrature understates the true SE — sometimes by 30–50% for long-tail books; (3) The bootstrap naturally captures this correlation because the same resampled residuals affect all AYs in each simulation — the bootstrap may be more reliable for portfolio SE than the Mack formula in practice.

B. Stochastic Methods (20 Questions)

q Compare the Mack model to the ODP Bootstrap. Which should you use?

Ideal Answer: Mack gives: analytical mean and SE; assumes nothing about the distribution shape; fast; good for interval estimates. ODP Bootstrap gives: full empirical reserve distribution; any risk measure (VaR, TVaR, CoV); both parameter and process uncertainty; computationally intensive. Use Mack when: you need quick interval estimates; computing power is limited; the audience only needs mean $\pm 2\sigma$. Use Bootstrap when: you need VaR for capital purposes; you need the reserve distribution shape; you need to compute the 99.5th percentile for SCR; IFRS 17 RA requires a specific confidence level. Both should agree on the mean — if bootstrap mean differs materially from CL, investigate. The SE should also be broadly consistent — bootstrap SE $\geq$ Mack SE (because bootstrap includes process error). Mack is the theoretical foundation; ODP Bootstrap is the practical implementation for capital and IFRS 17.

q What is TVaR (Tail Value at Risk) and why is it preferred to VaR for capital purposes?

Ideal Answer: $\text{TVaR}(p) = E[R \mid R \geq \text{VaR}(p)]$ — the expected reserve given that we're in the top $(1-p) \times 100\%$ of scenarios. $\text{VaR}(99.5\%)$ answers: "what is the 1-in-200 year loss?" $\text{TVaR}(99.5\%)$ answers: "given that we're in the worst 0.5% of scenarios, what is the average loss?" TVaR is preferred for capital because: (1) VaR is not subadditive — adding risks can create a higher capital requirement than diversified total, violating sound risk principles; (2) TVaR accounts for the shape of the tail, not just the threshold; if the tail is fat (high kurtosis), TVaR captures this; (3) For reinsurance sizing, TVaR gives the expected payout in the tail — more useful for treaty design; (4) Some regulators (Swiss Solvency Test) use TVaR rather than VaR. For IRDAI RBC purposes, the standard factor approach implicitly calibrates to a VaR measure; an internal model using TVaR would need justification.

q How would you handle a reserve valuation when the incremental triangle contains negative values?

Ideal Answer: Negative incrementals occur due to: reserve releases (case reserves set high then released), late salvage payments, or data errors. They cause three problems: (1) Chain ladder may produce LDFs < 1 (development going backwards), (2) ODP bootstrap requires positive values for Gamma simulation, (3) Log-linear GLM can't handle zeroes/negatives. Solutions: (1) Smooth/exclude: exclude the negative cell from LDF calculation; use medial (dropping high/low) or recent-year weighted factors. (2) Bornhuetter-Ferguson: BF is much less sensitive to negative incrementals than CL because the IBNR comes from the a priori, not the observed paid. (3) Incurred triangle: use incurred (paid + case) instead — IBNER-only development tends to be smoother. (4) GLM with adjustment: use a quasi-Poisson GLM that allows negatives in the quasi-likelihood framework. (5) Investigate: determine if negative is a genuine release (document and explain) or a data error (correct it). Negative incrementals on old accident years are generally benign; on young years they're a red flag.

q Explain the GLM-Chain Ladder equivalence. Why is it important?

Ideal Answer: England and Verrall (1999) showed that the standard chain ladder estimator is mathematically equivalent to a Poisson GLM fitted to the incremental triangle with accident year and development period as the only factors: $c(i,j) \sim \text{Poisson}(\alpha_i \times \beta_j)$, where α_i are accident year parameters and β_j are development period parameters. The VW LDF estimator is the MLE in this model. This equivalence is important because: (1) It gives the chain ladder a rigorous statistical foundation and enables proper residual analysis; (2) The GLM framework allows adding covariates (calendar year, exposure measures) that extend the basic model; (3) It enables formal statistical testing (LRT) to justify or reject each factor; (4) The overdispersed Poisson extension is the basis for the ODP Bootstrap; (5) The GLM formulation is what the BootChainLadder function in R implements internally. Understanding this equivalence distinguishes technically sophisticated candidates from those who just know "the recipe."

q How would you estimate the tail factor beyond the last observed development period?

Ideal Answer: Four main approaches: (1) Inverse power: $f(j)-1 = a/j^b$. Fit by regression on $\ln(f(j)-1)$ vs $\ln(j)$ for observed periods. Extrapolate and integrate to get CDF beyond the triangle. Robust and widely used. (2) Clark Loglogistic model: use Clark's maximum likelihood method with a loglogistic or Weibull development curve. The tail emerges naturally from the fitted curve rather than requiring explicit extrapolation. (3) Benchmark: use market data — Lloyd's of London Market Bulletin or a reinsurer's actuarial note for motor TP tail factors. Cross-check with internal long-tails where available. (4) Credibility blend: blend your estimated tail with the benchmark using credibility theory based on data volume. For IRDAI: tail factor selection must be documented and the appointed actuary must be satisfied it is reasonable. A common error is using a 1,000 tail (assuming triangle is fully developed) for a line like TP motor where development clearly continues — this understates IBNR materially.

C. IFRS 17 & Regulatory (20 Questions)

Q Under IFRS 17, what are the components of the Liability for Incurred Claims (LIC)?

Ideal Answer: Under IFRS 17 (Ind AS 117), the Liability for Incurred Claims represents the obligation for claims that have already occurred. It consists of: (1) Best Estimate Liabilities (BEL) — the probability-weighted expected present value of future cash flows. For non-life short-duration contracts, this is essentially the undiscounted IBNR (discounting is required for any claims expected to be paid more than 12 months from the reporting date). (2) Risk Adjustment (RA) — the compensation required for bearing the non-financial uncertainty beyond the BEL. Quantified at a specified confidence level and disclosed. (3) Discounting effect — for long-tail lines (TP motor, liability), claims paid many years hence must be discounted. The yield curve to use: risk-free rate from the local bond market. IRDAI's Ind AS 117 guidance (issued 2023–24) specifies the discount rate methodology. Under the PAA (Premium Allocation Approach) for short-duration contracts, discounting may be simplified. The key reserving output for IFRS 17 is the stochastic distribution — it provides both the BEL (mean) and the RA (confidence level quantile above mean).

Q What is the difference between the Risk Adjustment under IFRS 17 and the Solvency II risk margin?

Ideal Answer: Both capture uncertainty above the best estimate, but differ fundamentally in approach: IFRS 17 Risk Adjustment: (1) Entity-specific and principle-based — the company chooses its own method and confidence level; (2) Represents compensation for bearing risk — not prescriptive; (3) Must be disclosed with the equivalent confidence level; (4) Updated each reporting period; (5) Released as uncertainty resolves, creating a "risk adjustment release" that contributes to insurance service result. Solvency II Risk Margin: (1) Cost of Capital approach — mandated method; (2) $RM = 6\% \times SCR_{future}$, discounted over the run-off; (3) Objective: cost to transfer to a reference undertaking; (4) Calibrated at 99.5th percentile indirectly; (5) Very sensitive to yield curve changes — falling rates increase RM dramatically. Key point: IFRS 17 RA is higher quality information (reflects actual entity risk) but more subjective. Solvency II RM is mechanistic. IRDAI's Ind AS 117 guidance allows either a VaR-based or a cost of capital approach for the RA.

Q Your bootstrap shows a 99.5th percentile reserve of ₹568 Cr vs a standard formula SCR of ₹490 Cr. How do you reconcile this?

Ideal Answer: The discrepancy should first be understood, then a decision made on which is more reliable for capital purposes. Reasons for divergence: (1) The standard formula reserve risk module uses simplified volume-based factors — not calibrated to your specific development pattern. (2) The bootstrap is specific to your data; if your triangle is genuinely more volatile than average, the bootstrap should exceed the standard formula. (3) Standard formula factors are calibrated to industry averages — a long-tail line with unusual characteristics (court ruling exposure, PPO claimants) may have materially higher risk than standard. Steps: (1) Explain the difference in writing to the IRDAI/board — document that the internal model gives a higher capital requirement. (2) Consider whether a partial internal model for reserve risk is appropriate (IRDAI allows this under the RBC framework). (3) If the bootstrap is more reliable, management should consider using the higher figure for internal capital allocation. (4) Investigate: does the standard formula use the right volume measure? Does it apply the right line of business factor for motor TP?

Q What does the IRDAI require from the Appointed Actuary on reserve certification?

Ideal Answer: Under IRDAI's Insurance Act and related circulars, the Appointed Actuary (AA) must: (1) Issue an Actuarial Report and Certificate (ARC) for each financial year, certifying that the reserves are adequate to meet the insurer's obligations as they fall due; (2) The ARC must cover: recommended IBNR, IBNER, unexpired risk reserve (URR), and technical provisions; (3) The AA must comment on reserve uncertainty and any material reserving risks — stochastic reserving provides the evidence base; (4) The AA must independently assess the IBNR without being unduly influenced by management's preference for lower reserves; (5) Any divergence between the AA's recommended reserve and the management's carried reserve must be disclosed; (6) For IFRS 17 / Ind AS 117, the AA must certify the BEL, RA, and discounting methodology. The AA carries professional and regulatory liability for the reserve certification — a stochastic reserving framework with documented assumptions and sensitivity testing provides the best professional defence.

Q How would you present reserve uncertainty to a non-actuarial CFO?

Ideal Answer: I'd use three key messages: (1) The carried reserve of ₹420 Cr is our best estimate — meaning, on average, we'd expect to pay out around this amount. But claims are uncertain: about half the time we'll pay less, half the time more. (2) To be 90% confident we have enough set aside, we'd need to carry ₹484 Cr — that's an additional ₹64 Cr buffer. This is the cost of being more conservative. (3) In a catastrophic scenario (1-in-200 year), the reserve needed would be ₹568 Cr — that's the amount we need to hold as capital. I'd use a simple bar chart showing: minimum expected / best estimate / 75th percentile / 99.5th percentile as a "speedometer." Avoid technical jargon (Mack, ODP, bootstrap) — translate directly to business concepts. If the CFO asks why we don't just hold the 90th percentile all the time: that capital has a cost (returns forgone) — the target confidence level is a business decision balancing cost of capital against desired solvency security.

D. Advanced (20 Questions)

Q What is the Clark LDF method? When would you use it over chain ladder?

Ideal Answer: Clark (2003) fits a parametric development curve (loglogistic or Weibull) to the incremental triangle data using maximum likelihood. The expected incremental: $\mu(i,j) = \text{ELR} \times \text{Premium}(i) \times [G(j) - G(j-1)]$, where $G(j)$ is the cumulative development curve (e.g., loglogistic: $G(j) = j^\omega / (j^\omega + \theta^\omega)$, parameters ω, θ estimated by MLE). Advantages over chain ladder: (1) Parametric — tail emerges from the fitted curve, no ad hoc tail factor; (2) Smoothed — uses all data jointly, not just period-by-period columns; (3) Provides information matrix from MLE — direct confidence intervals; (4) Handles sparse data better — later periods share common shape parameter; (5) Clark's residuals follow a known distribution enabling formal testing. Use when: the development pattern is smooth and monotone (good for motor OD, property); you need a defensible tail without external benchmarks; you have sparse data in later development periods. Limitation: assumes a single development curve shape for all AYs — if the pattern shifts, Clark may miss it.

Q What is a Periodic Payment Order (PPO) and why does it create special reserving challenges?

Ideal Answer: A PPO (Periodic Payment Order, also called Structured Settlement) is a court-ordered arrangement where the injured party receives regular payments (often index-linked to earnings inflation) for the rest of their life, rather than a lump sum. This is common in severe UK motor TP and EL claims (brain injuries, spinal injuries). Reserving challenges: (1) Duration uncertainty: payments depend on the claimant's life expectancy — mortality assumptions are critical; (2) Inflation: payments are typically index-linked to ASHE (Annual Survey of Hours and Earnings), which is a care worker wage index — not general inflation; (3) Discounting: current claim reserve = PV of future annuity payments — reserve is very sensitive to discount rate (yield curve movements); (4) No settlement finality: a ₹500 Cr PPO has no fixed end date; reserve moves with mortality updates; (5) IBNR complexity: if some claimants will elect PPO at trial (currently receiving case reserve), the transition cannot be predicted from the paid triangle alone. Actuaries handling PPO books often use biometric models (mortality tables) combined with stochastic interest rate models — very different from standard chain ladder.

Q How would you test for calendar year effects in your triangle?

Ideal Answer: Three main approaches: (1) Graphical: plot standardised Mack residuals against calendar year (diagonal). If residuals in 2018–2019 are systematically positive and 2020–2021 negative, this suggests COVID was a calendar year effect (delayed settlements, backlog). (2) Venter (1994) test: formally test whether adding a calendar year parameter to the GLM chain ladder significantly improves fit. Compare the Poisson GLM with just $\alpha(i)$ and $\beta(j)$ factors against a model also including a diagonal parameter $\delta(k)$. Use a likelihood ratio test or AIC comparison. (3) Correlation test: within each diagonal k , compute the correlation between individual LDFs for adjacent development periods. High positive correlation → systematic effect is rotating all LDFs up or down together → calendar year effect. The Mack test statistic n_{plus} is derived from this. If calendar year effects are material: (a) include a diagonal

parameter in the GLM; (b) use the Clark method (no calendar year assumption); (c) if the effect is isolated (e.g., COVID 2020), consider adjusting the 2020 data or adding a one-off loading.

Q Describe the Bornhuetter–Ferguson as a credibility model. What is the credibility weight?

Ideal Answer: The BF method has a natural credibility interpretation. The BF ultimate is: $Ult_{BF} = C(i,J) + Q(i) \times (1 - 1/CDF) = Z \times C(i,J)/p + (1-Z) \times Q(i)$, where $Z = 1 - 1/CDF = \% \text{ unreported}$, and $p = 1/CDF = \% \text{ developed}$. This shows BF is a credibility blend: credibility $Z = \text{proportion of the ultimate that is still unreported (unobserved)}$. At $J = \text{first development period (20\% paid)}$: $Z = 0.80$ — 80% weight to a priori. At $J = \text{final period (95\% paid)}$: $Z = 0.05$ — 5% weight to a priori (essentially CL). The Bornhuetter (1972) paper frames this explicitly. The optimal credibility weight (from Bühlmann–Straub theory) is the value that minimises MSE. BF's weight $(1-1/CDF)$ is not generally optimal — it weights a priori too heavily in early periods for some lines. The Benktander iteration (one iteration of BF using the BF estimate as the new a priori) is empirically shown to be closer to optimal credibility. This is a favourite advanced question at UK qualified actuary interviews.

Q How would you estimate reserves for a brand new class of business with no historical triangle?

Ideal Answer: With no triangle: (1) Purely a priori: use the pricing actuary's loss ratio and apply industry development factors. Reserve = Premium $\times$ ELR $\times$ % unreported. (2) Industry analogues: obtain development triangles from similar portfolios (Lloyd's, GIC Re market data, reinsurer actuarial studies). Adjust development pattern for any known differences in settlement speed, claims handling. (3) Frequency–severity: estimate claim frequency (from exposure data and industry benchmarks) and average severity (from industry databases). Build bottom–up IBNR. (4) Bayesian hierarchical model: start with industry prior on LDFs; as first data emerges, update posterior. Bühlmann credibility formalises this. (5) Expert judgment: interview claims handlers and underwriters — their sense of notification delays and settlement durations provides a development pattern. The BF method shines here: it uses 100% a priori in the first development year, transitioning to observed data as the triangle develops. Document all assumptions, acknowledge high uncertainty, and commit to revisiting quarterly as experience emerges.

REMAINING 25 QUESTIONS – TOPIC AREAS

The remaining questions in the full 80 set cover: large claim separation and excess layer reserving; PPO/structured settlements; dynamic financial analysis (DFA) for reserve risk; stochastic BF; measurement of reserve uncertainty in reinsurance treaties; correlation between lines of business in aggregated reserve distributions; reserve risk in ORSA; Bayesian chain ladder; emerging claims and superimposed inflation; reserve margins and management judgement documentation; actuarial report structure; IRDAI vs IFRS 17

reconciliation; working with claims handlers on case reserve quality; and reserve experience monitoring.

14

GITHUB & LINKEDIN

Portfolio & GitHub Structure

Repository Structure

```
stochastic-reserving-portfolio/ |— README.md # Executive summary + key results |—
data/ | |— paid_triangle.csv # 10x10 paid claims triangle | |— incurred_triangle.csv
# 10x10 incurred triangle | |— premium.csv # Earned premium by AY | |—
reserve_distribution.npy # 10,000 bootstrap scenarios |— notebooks/ | |—
01_triangle_generation.ipynb # Synthetic data generation | |— 02_diagnostics_eda.ipynb
# LDF stability, LR development | |— 03_deterministic_cl_bf.ipynb # Chain Ladder + BF
| |— 04_mack_model.ipynb # Mack SE with full theory | |— 05_bootstrap_odp.ipynb # ODP
Bootstrap - full code | |— 06_reserve_distribution.ipynb # VaR, TVaR, CoV, IFRS 17 |
|— 07_validation.ipynb # Residuals, hindcast, sensitivity |— R/ | |—
chainladder_mack.R # ChainLadder package analysis | |— bootstrap_odp.R #
BootChainLadder R code | |— clark_ldp.R # Clark LDF parametric model |— src/ | |—
triangle_utils.py # Triangle manipulation helpers | |— mack.py # Mack model class |
|— bootstrap_odp.py # ODP Bootstrap class | |— risk_measures.py # VaR, TVaR, CoV,
IFRS 17 RA |— outputs/ | |— figures/ # All charts (ldf_stability, etc.) | |—
reserve_results.xlsx # CL + BF + Mack + Bootstrap table | |— actuarial_note.pdf #
Written actuarial report |— powerbi/ | |— reserving_dashboard.pbix # Power BI
dashboard file |— requirements.txt # Python: numpy, pandas, scipy, matplotlib |—
DESCRIPTION # R package dependencies |— .gitignore
```

LinkedIn Post Template

LINKEDIN POST — @YOUR_ACTUARIAL_GIRL



I built a complete Stochastic Reserving project — from first principles to IFRS 17 implications.

Here's what I found when I stopped at the chain ladder and went further:

- 📌 The chain ladder says: IBNR = ₹420 Cr. That's it. One number.
- 📌 Mack's model says: ₹420 Cr $\pm$ ₹42 Cr (CoV = 10.1%). Better.
- 📌 ODP Bootstrap says: here is a distribution of 10,000 possible futures — from ₹350 Cr to ₹590 Cr.
- ⚡ VaR(99.5%) = ₹568 Cr. That's ₹148 Cr more than the best estimate. That's capital.

The project covers everything from the triangle diagnostics to:

- ✅ Full Mack SE derivation with the cross-term formula (often missed!)
- ✅ ODP Bootstrap — all 5 algorithm steps with Python code
- ✅ IFRS 17 Risk Adjustment at 75th percentile
- ✅ Validation: residuals, hindcast back-testing, tail sensitivity
- ✅ 80 interview Q&As including the hard ones on Mack assumptions and GLM equivalence
- ✅ Full R code using the ChainLadder package

GitHub link in the comments. For IFoA CS2, CP1 and SP7/SP8 candidates — this is the project that shows you understand reserving, not just know it.

#ActuarialScience #Reserving #IFRS17 #IFoA #IAI #StochasticModelling

15 EXTENSIONS Advanced Extensions

Extension 1 — GLM Reserving (England & Verrall 1999)

PYTHON

```
# =====
# GLM Reserving – Poisson GLM equivalent to Chain Ladder
# Adds: calendar year effects, exposure variables, formal tests
# =====

import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import statsmodels.api as sm
```

```

# Reshape triangle to long format for GLM
def triangle_to_long(paid_df):
    """Convert upper triangle to long format for GLM"""
    rows = []
    n_ay = paid_df.shape[0]
    for i, ay in enumerate(paid_df.index):
        for j, dev_col in enumerate(paid_df.columns):
            val = paid_df.iloc[i, j]
            if not np.isnan(val):
                # Incremental claim for GLM
                incr = val - (paid_df.iloc[i, j-1] if j > 0 else 0)
                rows.append({
                    'AY': str(ay),
                    'Dev': str(j+1),
                    'CY': str(ay + j),    # calendar year diagonal
                    'incr_claims': max(incr, 0.1), # floor for Poisson
                })
    return pd.DataFrame(rows)

long_df = triangle_to_long(paid)

# — Model 1: Standard CL equivalent (no CY effects)
glm_base = smf.glm(
    'incr_claims ~ C(AY) + C(Dev)',
    data=long_df,
    family=sm.families.Poisson()
).fit()

# — Model 2: Add calendar year diagonal effects
# Tests Mack assumption M3: independence across accident years
glm_cy = smf.glm(
    'incr_claims ~ C(AY) + C(Dev) + C(CY)',
    data=long_df,
    family=sm.families.Poisson()
).fit()

# — Likelihood Ratio Test: Is CY effect significant?

```



```

lr_stat = 2 * (glm_cy.llf - glm_base.llf)
df_diff = glm_cy.df_model - glm_base.df_model
p_value = 1 - stats.chi2.cdf(lr_stat, df_diff)
print(f"LRT for CY effects:  $\chi^2$  = {lr_stat:.2f}, df = {df_diff}, p = {p_value:.4f}")
print(f"'CY effect significant – include in model' if p_value < 0.05 else 'No significant effect'")

# — ODP: quasi-Poisson for overdispersion correction
glm_odp = smf.glm(
    'incr_claims ~ C(AY) + C(Dev)',
    data=long_df,
    family=sm.families.NegativeBinomial() # alternative: quasi-Poisson not in statsmodels
).fit()

# GLM residuals for diagnostic plots
# Same as Section 10 residual checks, but now formal test available

```

Extension 2 — Stochastic BF (Verrall 2004)

The stochastic Bornhuetter–Ferguson (Verrall 2004) places a prior distribution on the expected ultimate loss ratio and derives a credibility-weighted reserve. The prior on ELR $\sim \text{Gamma}(\alpha, \beta)$ produces a posterior that is also Gamma, enabling closed-form MSE. This is implemented in the `ChainLadder` package via `BornhuetterFerguson(..., mse.method="Mack")`. The stochastic BF is more appropriate for immature accident years where the CL bootstrap produces implausibly wide distributions.

Extension 3 — Multi-Line Reserve Correlation

PYTHON

```

# Reserve correlation between lines of business
# Key for total company SCR: correlated reserves need aggregation formula
# Solvency II standard formula uses a correlation matrix C

# Example: Motor TP + Employer's Liability + Public Liability
#  $SCR_{total} = \sqrt{\sum_i \sum_j \rho_{ij} \times SCR_i \times SCR_j}$ 

CORR_MATRIX = np.array([
    [1.00, 0.50, 0.25], # Motor TP

```

```

    [0.50, 1.00, 0.50],    # Employer's Liability
    [0.25, 0.50, 1.00],    # Public Liability
]) # Source: Solvency II QIS5 reserve risk correlation matrix

SCR_by_line = np.array([14620, 8400, 5200]) # ₹ Lakhs

SCR_total_formula = np.sqrt(SCR_by_line @ CORR_MATRIX @ SCR_by_line)
SCR_independent    = np.sqrt(np.sum(SCR_by_line**2))

diversification_benefit = 1 - SCR_total_formula / SCR_by_line.sum()
print(f"Portfolio SCR (formula): ₹{SCR_total_formula:,.0f} Lakhs")
print(f"Diversification benefit: {diversification_benefit:.1%}")

# For internal model: use bootstrap simulation with correlated random seeds
# via Gaussian copula across line-specific bootstrap runs

```

Extension 4 — IFRS 17 Full Technical Provisions Calculation

PYTHON

```

# IFRS 17 Technical Provisions – Full Calculation
# TP = BEL + Risk Adjustment
# BEL = Discounted expected future cash flows

# — Discount rate (IRDAI/IFRS 17 risk-free rate) —————
# Use Indian Government Securities yield curve (December 2023)
DISCOUNT_RATES = {
    1: 0.0685, 2: 0.0700, 3: 0.0710,
    4: 0.0715, 5: 0.0718, 7: 0.0720, 10: 0.0725
}

def discount_cashflows(projected_payments, payment_lags, rates):
    """
    Discount projected claims payments by accident year and lag.
    Assumes simplified: payment mid-year of each development period.
    """
    bel = 0
    for (ay, lag), payment in zip(payment_lags, projected_payments):

```

```

# Discount factor: 1 / (1 + r)^lag (simplified; use spot rate)
r = rates.get(lag, rates[10]) # use 10yr rate for lags > 10
df = 1 / (1 + r) ** (lag - 0.5) # mid-period discount
bel += payment * df

return bel

# — Risk Adjustment —————
# Method: Confidence Level approach (most common for non-life)
# RA = VaR(75th percentile) - Best Estimate

best_estimate_undiscounted = np.mean(reserve_dist)
best_estimate_discounted = best_estimate_undiscounted * 0.962 # ~4% discount

RA_75pct = np.percentile(reserve_dist, 75) * 0.962 - best_estimate_discounted

print(f"IFRS 17 Technical Provisions:")
print(f"  BEL (discounted):      ₹{best_estimate_discounted:,.1f} Lakhs")
print(f"  Risk Adjustment (75%): ₹{RA_75pct:,.1f} Lakhs")
print(f"  Total TP:                ₹{best_estimate_discounted + RA_75pct:,.1f} Lakhs")
print(f"  RA / BEL:                {RA_75pct/best_estimate_discounted:.1%}")

```

Extension 5 — Back-Testing Framework

A rigorous back-test removes the latest n diagonals (typically 2–4 years), re-runs the entire stochastic analysis, and compares the predicted distribution against what actually emerged. Key metrics: (1) Was the actual reserve within the predicted 95% CI? (2) Was the actual reserve above or below the median? (3) Track this over rolling 4-quarter windows to detect systematic bias. If actual consistently exceeds the median → systematic under-reserving. This is the most powerful evidence of model adequacy and is expected by sophisticated reviewers (Lloyd's, SOA, IFoA) when reviewing the actuarial report.

Extension 6 — Superimposed Inflation Modelling

Superimposed inflation (SI) is claims cost inflation above general price inflation — driven by court awards (Ogden rate changes), medical cost inflation, legal fee increases. For long-tail TP lines, SI can be 3–5% per annum above CPI. Modelling SI explicitly: fit an exponential trend to calendar-year diagonal averages (after adjusting for mix shifts). Apply the estimated annual SI rate to future projected payments. Bootstrap the SI estimate as an additional source of

uncertainty. This is a significant risk for motor TP that is often inadequately captured by the standard chain ladder.

Stochastic Reserving

ACTUARIAL PORTFOLIO PROJECT · SENIOR ANALYST LEVEL

Motor TP liability · Mack · ODP Bootstrap · IFRS 17
Synthetic data only — ShieldRe is a fictional insurer